

UNIVERSIDAD POLITÉCNICA DE MADRID

ESCUELA TÉCNICA SUPERIOR
DE INGENIEROS DE TELECOMUNICACIÓN



GRADO EN INGENIERÍA Y SISTEMAS DE DATOS

TRABAJO FIN DE GRADO

APLICACIÓN DE TÉCNICAS DE MACHINE LEARNING PARA LA
IDENTIFICACIÓN Y OPTIMIZACIÓN DE PATRONES EN 'SWING CHART' EN
MERCADOS FINANCIEROS

Lijun Zhou

2026

Grado en Ingeniería y Sistemas de Datos

TRABAJO DE FIN DE GRADO

Título: Aplicación de técnicas de Machine Learning para la identificación y optimización de patrones en 'Swing Chart' en mercados financieros

Autor: D. Lijun Zhou

Supervisión técnica:

Supervisión académica: Dr. Eduardo López Gonzalo

Departamento: Departamento de Señales, Sistemas y Radiocomunicaciones.

MIEMBROS DEL TRIBUNAL

Presidente:

Vocal:

Secretario:

Suplente:

Los miembros del tribunal arriba nombrados acuerdan otorgar la calificación de:

.....

Fecha de lectura:

UNIVERSIDAD POLITÉCNICA DE MADRID

ESCUELA TÉCNICA SUPERIOR
DE INGENIEROS DE TELECOMUNICACIÓN



GRADO EN INGENIERÍA Y SISTEMAS DE DATOS

TRABAJO FIN DE GRADO

APLICACIÓN DE TÉCNICAS DE MACHINE LEARNING PARA LA
IDENTIFICACIÓN Y OPTIMIZACIÓN DE PATRONES EN 'SWING CHART' EN
MERCADOS FINANCIEROS

Lijun Zhou

2026

Resumen

El presente proyecto tiene como objetivo el desarrollo e implementación de un sistema basado en técnicas de Machine Learning orientado al análisis y detección de patrones en “Swing Charts”, una herramienta ampliamente utilizada en el análisis técnico de los mercados financieros. A partir de datos históricos y en tiempo real, el sistema generará representaciones de “Swing Charts” a partir de “Bar Charts” y aplicará algoritmos de aprendizaje supervisado y no supervisado para identificar configuraciones recurrentes, como dobles máximos, dobles mínimos o rupturas de tendencia.

El desarrollo se llevará a cabo principalmente en Python, utilizando librerías como pandas, scikit-learn y matplotlib, además de entornos de conexión con plataformas de trading como Interactive Brokers API (IBKR). El proyecto busca evaluar la capacidad predictiva de los modelos entrenados y su utilidad en la generación de señales de compra o venta dentro de una estrategia de trading algorítmico. Con ello, se pretende aportar un enfoque innovador que combine la solidez del análisis técnico clásico con la adaptabilidad de las técnicas modernas de inteligencia artificial.

Palabras clave

Trading algorítmico, predicción de mercados financieros, automatización, análisis en tiempo real, backtesting.

Summary

XXXXXXXXX

Keywords

XXXXXXXXX

Índice

1	Introducción y objetivos	1
1.1	Introducción	1
1.2	Objetivos	2
2	Estado del arte	5
2.1	Tendencias en el trading algorítmico	5
2.2	Modelos para series temporales: RNN, LSTM y GRU	5
2.2.1	LSTM	5
2.2.2	GRU	6
2.3	Tendencias recientes en el trading cuantitativo	7
2.4	Comparativa entre indicadores tradicionales, ML clásico, ML profundo y IA agéntica	7
3	Desarrollo	9
3.1	Pipeline	9
3.2	Desarrollo del autómata	11
3.2.1	Obtención de datos de los futuros	12
3.2.2	Carga y limpieza	13
3.2.3	Cálculo de características	15

3.2.4	Selección de características	17
3.2.5	Definición y cálculo de la clase objetivo	17
3.2.6	Aprendizaje supervisado	18
3.2.7	Aprendizaje no supervisado	22
3.2.8	Análisis de resultados de entrenamiento	23
3.2.9	Backtest de las predicciones	25
3.2.10	Simulación de Monte Carlo	27
3.2.11	Gestión de cartera	27
3.2.12	Análisis de resultados de optimización y Backtest de las optimizaciones y Simulación de Monte Carlo	29
3.2.13	Manejo del riesgo	30
3.2.14	Operación	30
3.2.15	Logging	32
3.3	Despliegue usando contenedores de Docker	33
3.3.1	Configuración del contenedor principal	33
3.3.2	Bases de datos	34
3.3.3	Visualización	36
3.3.4	Alertas y avisos	37
3.3.5	Orquestación y MLOps	37
4	Resultados	39
4.1	Resultados	39
5	Conclusiones y líneas futuras	43
5.1	Conclusiones	43
5.2	Líneas futuras	43

6 Aspectos éticos, económicos, sociales y ambientales	47
6.1 Introducción	47
6.2 Descripción de impactos relevantes relacionados con el proyecto	47
6.3 Análisis detallado de alguno de los principales impactos	47
6.4 Conclusiones	47
7 Presupuesto económico	48
Bibliografía	49
Anexos	50
A Resultados de los entrenamientos	50
B Resultados de los backtest individuales	50
C Repositorio de GitHub con el código	50

Listado de figuras

2.1	Arquitectura LSTM, [1]	6
2.2	Arquitectura GRU, [1]	6
3.1	Añadir img con el diagrama de cómo funciona todo	12
3.2	Comparación antes y después de ajustar	14
3.3	Añadir img con algunos plots de las características más usadas como modelos solos	15
3.4	Añadir img de las capas de las redes LSTM	19
3.5	Añadir img de las capas de las redes GRU	19
3.6	Añadir img de walk forward	21
3.7	Resultados obtenidos por el modelo LSTM.	23
3.8	Resultados obtenidos por el modelo GRU.	24
3.9	Añadir img con evolución de las métricas dependiendo de los umbrales	25
3.10	Añadir img con una simulación del Backtest	25
3.11	Añadir img con una simulación de Monte Carlo con los percentiles	27
3.12	Añadir img con Backtest	29
3.13	Añadir img con una simulación de Monte Carlo	29
3.14	Añadir img con el diagrama de cómo funciona todo, los puertos usados, etc	33

3.15	Añadir img dashboard grafana	36
3.16	Añadir img Alertmanager o Slack	37
4.1	Añadir img con equity curve, max drawdown y sharpe ratio	39
4.2	Añadir img comparativa del Backtest vs Robotrader	40
4.3	Añadir img comparativa del Monte Carlo vs Robotrader	40
4.4	Añadir img comparativa del Monte Carlo vs Robotrader	40
4.5	Añadir img comparativa del Monte Carlo vs Robotrader	41
5.1	Añadir img con algunos plots de las características más usadas como modelos solos	45
5.2	Añadir img con algunos plots de las características más usadas como modelos solos	45
5.3	Añadir img con algunos plots de las características más usadas como modelos solos	46
5.4	Añadir img con algunos plots de las características más usadas como modelos solos	46

Listado de tablas

1.1	Características principales de los contratos de futuros analizados	4
3.1	Ejemplo de registro histórico OHLCV de un contrato de futuros	13
3.2	Tabla comparativa de parámetros de los modelos	18
3.3	Cuadrícula de modalidades operativas según uso de scale-in y sizing dinámico. .	26
3.4	Cuadrícula de modalidades operativas según uso de scale-in y sizing dinámico. .	26
3.5	Tabla comparativa de gestión de riesgo	28
3.6	Estructura de la tabla <code>metrics</code>	34
3.7	Estructura de la tabla <code>risk_events</code>	34
3.8	Estructura de la tabla <code>signals</code>	34
3.9	Estructura de la tabla <code>trades</code>	35
4.1	Tabla resumen de los 13 futuros evaluados en las cuatro modalidades operativas. Desde 2026-01-20 hasta 2026-06-01	42
4.2	Tabla resumen de los 13 futuros evaluados en las cuatro modalidades operativas. Desde 2026-03-27 hasta 2026-06-01, aunque Robotrader empezó el 6 para tener datos para las primeras predicciones	42

Todo list

☐	Revisar	12
☐	Sí se generaliza por mercados -6 y +1	14
☐	Explicar mejor cómo funciona la función de inferencia, decidir si pasar a UTC o no	14
☐	Hacer alusión al libro	16
☐	definir el reto: lista de todos los targets posibles además de long short nada, otros de clasificación, regresión, etc y lo que conlleva cada uno	18
☐	Definir los swings, y las estructuras que más se usan	18
☐	Se me ha olvidado qué quería poner de más	19
☐	Explicar el tema de los umbrales	19
☐	Explicar entrenamiento	19
☐	Faltan cosas por explicar	22
☐	Añadir curva auc roc para las dos capas de clasificación	24
☐	Añadir curva loss por fold	24
☐	Ver si se puede meter un contador con racha de victorias y derrotas	25
☐	Explicar qué es Pyomo	28
☐	Implementar la gestión de cartera	28
☐	Comparar respecto a usar todo el capital en un solo activo	29
☐	Detallar las tablas	34

■ Tanto Airflow como MLflow, ver si hay tiempo para implementarlos	37
■ Añadir grafica de la evolución del equity, comparativa contra Robotrader	41
■ Añadir Monte Carlo	41
■ Preguntar si demasiadas lineas futuras demuestra poco o mucho interes en el trabajo	43

Glosario

OHLCV - Precios Open High Low y Close y Volumen

XXXX - XXXXXXXXX

Introducción y objetivos

1.1 Introducción

El trading financiero tradicional presenta una limitación fundamental, una fuerte dependencia de los criterios subjetivos, los sesgos cognitivos y el estado emocional de cada inversor. Factores como el miedo o la euforia pueden conducir a decisiones inconsistentes y alejadas de una gestión óptima del riesgo. Para mitigar este problema surgió el *trading algorítmico*, un enfoque donde las decisiones se toman en base a reglas matemáticas predefinidas, eliminando en gran medida la intervención humana durante la operativa.

La evolución natural del *trading algorítmico* ha sido la incorporación de modelos de aprendizaje automático (ML) que se entrenan usando un conjunto de características generadas a partir de datos históricos de los diferentes activos financieros a operar. Estos modelos permiten aprender patrones complejos a partir de datos históricos, capturar relaciones no lineales y adaptarse a diferentes regímenes de mercado. El presente trabajo se centra en esta línea, el diseño y desarrollo de un sistema de trading basado en ML capaz de operar de forma autónoma sobre un conjunto diverso de futuros financieros.

Para la elaboración de este trabajo se toman como referencia dos obras. Por un lado, *Trading Algorítmico con Python* de Isaac Trullàs [2], que proporciona una guía sobre el flujo de trabajo completo para el desarrollo de una estrategia algorítmica, desde la carga y limpieza de los datos, pasando por el cálculo y validación de las características que alimentarán a los modelos hasta la evaluación final mediante el cálculo de métricas para comprobar que la estrategia desarrollada es consistente, robusta y capaz de generar beneficios de forma sostenida. Por otro lado, *Safety in the Market* de David E. Bowden[3], que introduce conceptos fundamentales de análisis de mercado y del trading.

En los mercados financieros existen diferentes tipos de activos, cada uno con sus características, con diferentes niveles de riesgo y con comportamientos distintos frente a las condiciones económicas. Entre los más comunes se encuentran las acciones, que representan participaciones dentro de una empresa; los bonos, que constituyen instrumento de deuda emitidos por gobiernos o corporaciones; las materias primas, como el oro, petróleo o productos agrícolas; y las divisas, cuyo valor se basa en el tipo de cambio que existe entre monedas. Por otra parte existen derivados financieros, cuyo valor depende de un activo subyacente. Dentro de esta categoría se encuentran los futuros, contratos estandarizados que permiten tomar posiciones largas o cortas sobre índices bursátiles, divisas, materias primas o renta fija. Una posición larga significa comprar barato esperando que el precio aumente para vender, mientras que una posición corta consiste en vender anticipadamente esperando que el precio disminuya para recomprar más barato. Existe una asimetría en el riesgo asociado a cada tipo de posición, la

ventaja de las posiciones a largo reside en que en caso de que el precio disminuya, solo puede caer hasta 0 mientras que en las posiciones cortas el precio puede aumentar sin un techo, haciendo especialmente relevante la gestión de riesgo.

En este trabajo se ha optado la operación sobre futuros en vez de sobre otros activos por diversas razones.

- **Mercados regulados y centralizados:** Las negociaciones se realizan en mercados regulados y centralizados (CME, MEFF, entre otros), garantizando transparencia y control, disminuyendo los riesgos de manipulación.
- **Alta liquidez:** Ofrecen una elevada liquidez, facilitando la ejecución eficiente de órdenes incluso en estrategias algorítmicas.
- **Estandarización de los contratos:** Todos los contratos comparten especificaciones fijas —tamaño, tick, vencimiento, márgenes— lo que simplifica la obtención, comparación y tratamiento de los datos.
- **Uso de márgenes y apalancamiento:** El uso de márgenes permite operar con apalancamiento, aumentando el potencial de beneficio aunque también el riesgo. Por ello, la gestión del riesgo es fundamental, como se detallará más adelante. Este mecanismo permite controlar posiciones mayores con un capital limitado.
- **Rollover:** Existe la posibilidad de realizar *rollover*, evitando la entrega física del activo y permitiendo mantener la operativa de forma continua.

1.2 Objetivos

El objetivo principal de este presente trabajo es desarrollar un sistema capaz de operar de manera autónoma una cartera con un valor de un millón de dólares, maximizando el beneficio mientras asume el menor riesgo posible. Como se puede observar en la Tabla 1.1, el sistema operará sobre 13 futuros distintos que pertenecen a distintas categorías, divisas, materias primas, índices bursátiles y bonos del Tesoro estadounidense, proporcionando así una exposición diversificada a diferentes clases de activos y dinámicas de mercado.

A continuación se describen los futuros sobre los que se llevará a cabo la operativa.

AD - Australian Dollar

Futuro sobre el dólar australiano frente al dólar estadounidense. Muy utilizado en estrategias macro y ligadas a materias primas.

BP - British Pound

Futuro sobre la libra esterlina. Alta liquidez y relevancia en estrategias FX y cobertura frente a decisiones del Banco de Inglaterra.

CL - Crude Oil WTI

Futuro del petróleo WTI. Uno de los contratos más negociados del mundo, con elevada volatilidad y fuerte sensibilidad geopolítica.

EC - Euro FX

Futuro del euro frente al dólar. El contrato FX más líquido del CME, adecuado para estrategias direccionales y de reversión.

ES - E-mini S&P 500

Futuro del índice S&P 500 en formato reducido. Altísima liquidez y representatividad del mercado estadounidense.

GC - Gold

Futuro del oro. Activo refugio con dinámicas propias en periodos de incertidumbre y movimientos de tipos reales.

MFXI - Micro Euro FX (M6E)

Versión micro del futuro del euro. Permite exposición granular con menor margen, ideal para calibración fina.

NG - Natural Gas

Futuro del gas natural. Muy volátil y con fuerte estacionalidad, relevante en estrategias de energía.

NQ - E-mini Nasdaq 100

Futuro del Nasdaq 100. Más volátil que el ES, con fuerte exposición al sector tecnológico.

YM - E-mini Dow Jones

Futuro del Dow Jones Industrial Average. Menor volatilidad y comportamiento más estable que NQ y ES.

ZB - 30Y US Treasury Bond

Futuro del bono del Tesoro a 30 años. Muy sensible a expectativas macroeconómicas de largo plazo.

ZN - 10Y US Treasury Note

Futuro del Treasury a 10 años. El contrato de renta fija más líquido del mundo, referencia clave en estrategias macro.

ZS - Soybean Futures

Futuro de la soja. Presenta estacionalidad marcada y sensibilidad a factores climáticos y agrícolas globales.

Tabla 1.1: Características principales de los contratos de futuros analizados

Símbolo	Nombre	Mult.	Tick Size	Tick Value	Margen Init
AD	Australian Dollar	100000	0.00005	\$5	\$2503.59
BP	British Pound	62500	0.0001	\$6.25	\$2585.50
CL	Crude Oil WTI	1000	0.01	\$10	\$28336.53
EC	Euro FX	125000	0.00005	\$6.25	\$3671.13
ES	E-mini S&P 500	50	0.25	\$12.5	\$32380.88
GC	Gold	100	0.1	\$10	\$44136.61
MFXI	Micro Euro FX (M6E)	12500	0.0001	\$1.25	\$367.60
NG	Natural Gas	10000	0.001	\$10	\$6592.86
NQ	E-mini Nasdaq 100	20	0.25	\$5	\$60368.33
YM	E-mini Dow Jones	5	1	\$5	\$17296.56
ZB	30Y US Treasury Bond	1000	0.03125	\$31.25	\$4259.66
ZN	10Y US Treasury Note	1000	0.015625	\$15.625	\$2156.85
ZS	Soybean Futures	5000	0.0025	\$12.5	\$4077.58

Para alcanzar este objetivo se ha diseñado una arquitectura basada en modelos de aprendizaje automático que asisten en la toma de decisiones operativas. La propuesta inicial contempla el uso combinado de modelos de aprendizaje supervisado y no supervisado, cuyos papeles se detallará en los siguientes capítulos. En términos generales, los modelos de aprendizaje profundo se emplearán como modelos supervisados para la generación de señales de trading, mientras que los modelos no supervisados se utilizarán para identificar regímenes de mercado y agrupar comportamientos similares mediante técnicas de clustering.

La arquitectura propuesta se concibe, por tanto, como un sistema híbrido: los modelos supervisados proporcionan señales direccionales, y los modelos no supervisados aportan información contextual que puede mejorar la robustez y la interpretación del entorno de mercado.

No obstante, aunque esta sea la intención inicial, la implementación final puede diferir debido a limitaciones encontradas durante el desarrollo, cuyas motivaciones serán explicadas en los capítulos correspondientes. En cualquier caso, la arquitectura mantiene la capacidad de integrar modelos no supervisados en futuras ampliaciones.

Finalmente, el objetivo último de este trabajo es poder poner el uso la mayor cantidad de conocimiento obtenido de estos años de estudio en un solo trabajo, incluyendo principalmente programación en Python, despliegue del trabajo en Docker, entrenamiento de modelos de Machine Learning, optimización y diseño de pipelines de datos.

Estado del arte

2.1 Tendencias en el trading algorítmico

El trading algorítmico ha experimentado una evolución significativa en las últimas décadas, pasando de estrategias basadas exclusivamente en indicadores técnicos a sistemas que integran técnicas avanzadas de aprendizaje automático y modelos de aprendizaje profundo. El estado del arte actual combina ingeniería de datos, ingeniería de características, modelado predictivo, análisis cuantitativo y técnicas de optimización, con el objetivo de capturar patrones complejos en series temporales financieras altamente ruidosas y no estacionarias.

2.2 Modelos para series temporales: RNN, LSTM y GRU

Las Redes Neuronales Recurrentes (RNN) clásicas [4] fueron diseñadas para modelar dependencias temporales, pero presentan el problema del desvanecimiento del gradiente. Durante la retropropagación, los gradientes se atenúan al multiplicarse repetidamente por derivadas de funciones de activación como *sigmoid* o *tanh*, lo que provoca que las capas iniciales reciban gradientes casi nulos y solo se aprendan dependencias de corto plazo.

Para resolver esta limitación se propusieron dos arquitecturas mejoradas: LSTM (Long Short-Term Memory) [5] y GRU (Gated Recurrent Unit) [6]. Ambas introducen mecanismos de compuertas que regulan el flujo de información y permiten capturar dependencias de largo plazo de manera estable.

2.2.1 LSTM

Las LSTM introducen una memoria interna (*cell state*) y tres compuertas (entrada, olvido y salida) que controlan qué información se mantiene y cuál se descarta, permitiendo modelar dinámicas temporales complejas.

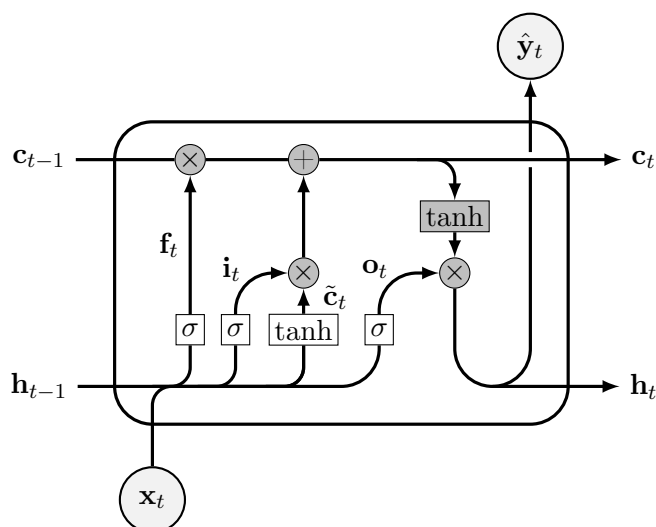


Figura 2.1: Arquitectura LSTM, [1]

2.2.2 GRU

Las GRU simplifican esta estructura mediante dos compuertas (actualización y reinicio), reduciendo el número de parámetros y acelerando el entrenamiento, manteniendo un rendimiento comparable al de las LSTM.

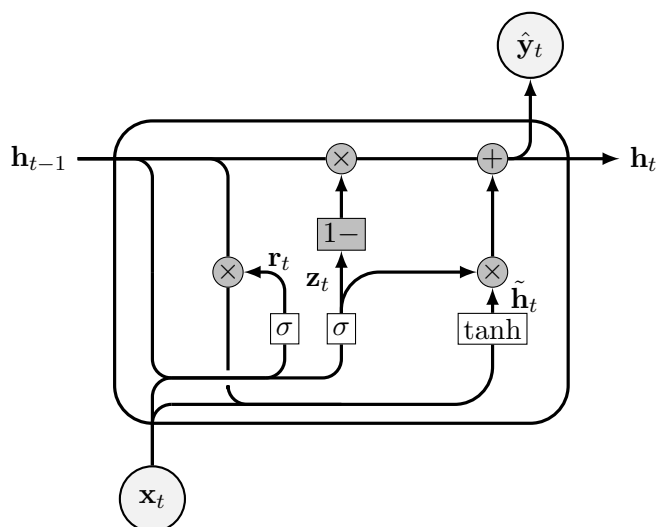


Figura 2.2: Arquitectura GRU, [1]

Ambas arquitecturas han demostrado ser especialmente útiles en series temporales financieras

debido a su capacidad para modelar dinámicas no lineales y dependencias temporales complejas y patrones ruidosos característicos de los mercados.

A pesar de su capacidad para capturar relaciones no lineales y dependencias temporales complejas, los modelos de aprendizaje profundo presentan un problema fundamental: su falta de interpretabilidad. Estas arquitecturas actúan como cajas negras, ya que no ofrecen una explicación clara de los factores que conducen a una predicción concreta. Esta opacidad dificulta comprender qué patrones del mercado está utilizando el modelo.

2.3 Tendencias recientes en el trading cuantitativo

Las tendencias recientes en el trading cuantitativo incluyen

- **Modelos basados en Transformers** para series temporales, capaces de capturar dependencias largas mediante mecanismos de atención.
- **Meta-learning** y técnicas de adaptación rápida a nuevos activos o regímenes de mercado.
- **Modelos híbridos** que combinan indicadores tradicionales con aprendizaje profundo.
- **Detección de regímenes de mercado** mediante clustering no supervisado para contextualizar las señales de trading.
- **Estudios de ablation** para evaluar la contribución real de cada componente del sistema y evitar sobreajuste.

2.4 Comparativa entre indicadores tradicionales, ML clásico, ML profundo y IA agéntica

Los enfoques utilizados en trading cuantitativo han evolucionado de manera significativa en las últimas décadas. Tradicionalmente, las estrategias se basaban en indicadores técnicos, como medias móviles, RSI, MACD o ATR, que transforman el precio en señales determinísticas. Aunque estos indicadores son sencillos de interpretar y computacionalmente eficientes, presentan limitaciones importantes: dependen de parámetros fijos, no capturan relaciones no lineales y suelen fallar en entornos de mercado cambiantes o altamente volátiles.

El siguiente paso en esta evolución fue la incorporación de modelos de aprendizaje automático clásico, como árboles de decisión, Random Forest, SVM o regresiones penalizadas. Estos modelos permiten aprender relaciones más complejas entre múltiples variables y generalizan mejor

que los indicadores tradicionales. Sin embargo, siguen teniendo dificultades para capturar dependencias temporales profundas y requieren un diseño manual de características (feature engineering) para representar adecuadamente la dinámica del mercado.

Con el auge del cómputo acelerado y la disponibilidad de grandes volúmenes de datos, surgieron los modelos de aprendizaje profundo, especialmente arquitecturas como LSTM, GRU, CNN 1D y, más recientemente, Transformers. Estos modelos son capaces de aprender patrones no lineales, dependencias de largo plazo y estructuras temporales complejas directamente a partir de los datos, reduciendo la necesidad de ingeniería manual. Su principal desventaja es la falta de interpretabilidad, ya que funcionan como cajas negras, y su tendencia al sobreajuste si no se aplican técnicas adecuadas de regularización y validación temporal.

Finalmente, en los últimos años ha emergido el concepto de IA agentica aplicada al trading, donde agentes autónomos combinan modelos predictivos, razonamiento secuencial y toma de decisiones basada en objetivos. Aunque este enfoque es prometedor, su uso en entornos financieros reales es todavía limitado debido a la complejidad de su validación, la dificultad para garantizar estabilidad y la necesidad de mecanismos robustos de control del riesgo.

Desarrollo

3.1 Pipeline

El desarrollo del sistema se estructura en varias fases dentro del proceso ETL (Extract, Transform, Load). En primer lugar, se realiza la carga de los datos históricos de los distintos contratos de futuros que se van a operar. A continuación, estos datos se someten a un proceso de limpieza y filtrado para eliminar valores erróneos, inconsistencias y posibles fuentes de ruido. Posteriormente, se lleva a cabo la transformación de los datos en un conjunto de características que servirán como entrada para los modelos de aprendizaje automático. Una vez entrenados los modelos, se evalúa su rendimiento mediante diferentes métricas y utilizando datos históricos no vistos para comprobar su capacidad de generalización. Finalmente, el sistema entra en fase operativa, donde procesa datos en tiempo real y genera señales de trading de manera autónoma.

El flujo de trabajo se organiza siguiendo la arquitectura medallón, donde los datos atraviesan por tres capas diferenciadas:

Bronce :

Contiene los datos históricos y en vivo en su forma cruda, tal y como se obtienen de las fuentes originales.

Plata :

En esta capa los datos se limpian, se validan, se filtran y se normalizan, garantizando su coherencia y calidad antes de ser utilizados.

Oro :

Los datos se transforman en *features* y *targets* listos para el entrenamiento de los modelos supervisados y para la inferencia en tiempo real.

A continuación muestro un mapa conceptual de las diferentes etapas y los hitos más importantes que se van a desarrollar en este trabajo.

1. **Carga y estandarización de datos:** descarga de datos minuto a minuto y unificación temporal a UTC.
2. **Limpieza y validación:** eliminación de datos erróneos, filtrado de fines de semana y verificación de coherencia.
3. **Enriquecimiento temporal:** asignación del día de negociación y resampleo a intervalos de 4 horas.

4. **Cálculo de variables derivadas:** generación de datos acumulados y transformaciones básicas.
5. **Ingeniería de características:** construcción de *features* avanzadas y análisis para evitar *data leakage*.
6. **Selección de características:** filtrado manual y mediante modelos tradicionales con mayor explicabilidad.
7. **Definición y cálculo de los *targets*:** construcción de las variables objetivo para los modelos supervisados.
8. **Entrenamiento y evaluación:** ajuste de hiperparámetros, búsqueda de umbrales óptimos y evaluación con métricas de clasificación, regresión y métricas financieras.
9. **Preparación para despliegue:** guardado de modelos, *scalers*, configuración y lista final de *features*.
10. **Obtención de información actualizada de los futuros:** recopilación de márgenes iniciales y de mantenimiento, multiplicadores, *tick size*, *tick value*, horarios de negociación, spread estimado y slippage medio.
11. **Backtesting y simulación:** ejecución de backtests sobre datos históricos y simulaciones Monte Carlo para evaluar robustez y sensibilidad a escenarios adversos.
12. **Operativa y monitorización:** ejecución del sistema, optimización de la cartera, control de riesgos, monitorización continua y análisis de rendimiento.
13. **KPI:** dockerización del sistema, integración con Prometheus, Loki y SQL para métricas y logs, visualización mediante grafana
14. **Alertas:** Vigilancia de las métricas y logs que al superar umbrales definidos envían alertas a Slack a través de Alertmanager

El sistema operará en interdía ya que este marco temporal ofrece diferentes ventajas para sistemas de aprendizaje automático. En primer lugar, los datos presentan menor ruido microestructural, lo que facilita el aprendizaje de patrones relevantes y disminuye el riesgo de *overfitting*. Además, los modelos no sufren por la alta no estacionalidad y los cambios de volatilidad que existen en los datos intradía. A nivel operativo, el trading interdía requiere menor monitorización continua de la situación, dado que se ejecutan menos operaciones y se mantiene las posiciones durante varios días o incluso semanas. Por otro lado, las estructuras de precio basadas en swings son más claras y consistentes en marcos interdía, lo que las hace especialmente adecuadas para la extracción de características y el análisis estructural del mercado.

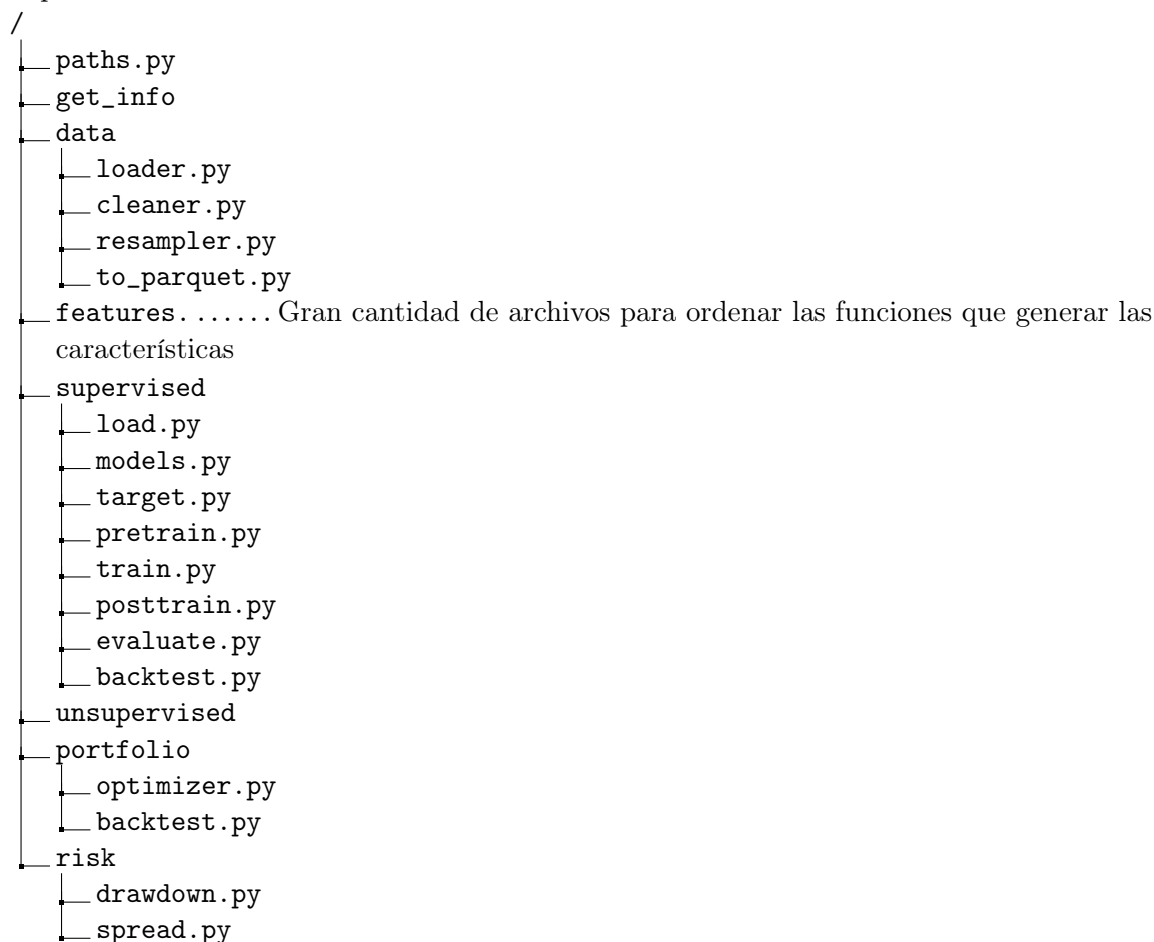
3.2 Desarrollo del autómata

Uno de los objetivos principales del proyecto es garantizar que el sistema sea modular, reutilizable y fácilmente extensible, independientemente del activo financiero con el que se trabaje.

Para ello, se ha diseñado una arquitectura que separa claramente las responsabilidades y permite incorporar nuevos componentes sin modificar el núcleo del sistema.

Con este propósito, la lógica del código se ha implementado dentro de un paquete instalable de Python, utilizando la instalación en modo editable mediante `pip install -e`. Esta decisión facilita el desarrollo iterativo, permite mantener una estructura coherente del proyecto y asegura que cualquier módulo pueda importarse desde otros scripts o notebooks sin necesidad de duplicar código.

La organización del paquete sigue una estructura modular que agrupa las funcionalidades por dominios lógicos, permitiendo añadir nuevos activos, modelos o pipelines de datos sin alterar la arquitectura existente.



```

├── execution
│   ├── broker.py
│   ├── live.py
│   ├── orders.py
│   ├── positions.py
│   └── killswitch.py
├── logging
│   └── logeer.py
└── retrain
    
```

Revisar

El orden mostrado en la imagen superior representa el orden lógico en el que se va llamando a los diferentes trozos de código.

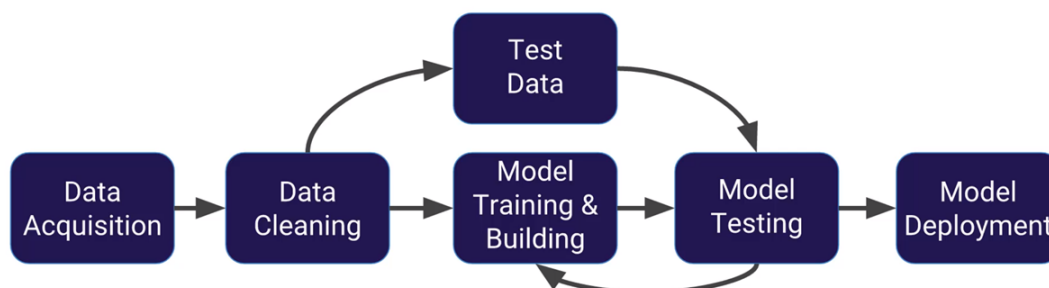


Figura 3.1: Añadir img con el diagrama de cómo funciona todo

Hay funciones que están en algunos de los directorios que pueden servir para varias etapas del proceso, principalmente aquellas encargadas de obtener los datos desde los archivos.

El código se puede revisar en el enlace referenciado en el Anexo C

A continuación paso a explicar cada una de las fases del flujo de los datos y las funciones utilizadas en ellas.

3.2.1 Obtención de datos de los futuros

Se me han facilitado los datos históricos OHLCV de distintos futuros con los que voy a operar. Pero existen diferentes datos que no se me han proporcionado, por ejemplo el mercado donde se opera cada futuro o la moneda con la que operan, estos dos datos son necesarios para posteriormente poder obtener el resto de información. Una vez obtenido estos dos datos nos podemos conectar mediante a API de Interactive Brokers al sistema y mediante una petición

obtener el resto de información. La información se divide entre la información estática y la variable.

Dentro de la información estática o casi estática ya que no suele cambiar está, además de lo mencionado anteriormente se encuentra el identificador de cada futuro, el apalancamiento implícito, el tamaño mínimo de movimiento y el valor de este movimiento.

La información dinámica, cambia cada día o semana, estos datos son las horas de apertura aunque siguen una regla general pero cada bolsa tiene sus festivos dependiendo del país donde estén, esta información nos puede servir para saber si podemos operar o no con cierto futuro. También hay información de las horas más líquidas aunque en nuestro caso usaremos estadísticas derivadas de los datos para decir qué horas son más líquidas para disminuir el slippage, es la diferencia de precio entre el precio esperado al enviar una orden y el precio real al que se ejecuta la orden. Y por último también tenemos dos datos muy importantes, que son los márgenes, el inicial y el de mantenimiento. El margen inicial es la cantidad mínima de dinero necesaria para abrir una posición y el de mantenimiento es el que pide el broker para que pueda mantener la posición abierta, este margen es mayor en el trading interdía para poder resistir los posibles saltos de precio al abrir el mercado, llamado Price Gap u Opening Gap.

3.2.2 Carga y limpieza

Se me han facilitado los datos históricos OHLCV de distintos futuros con los que voy a operar.

Tabla 3.1: Ejemplo de registro histórico OHLCV de un contrato de futuros

Ticker	Per	Fecha	Hora	Open	High	Low	Close	Volumen	OpenInt
AD	I	20010502	060100	0.52000	0.52000	0.52000	0.52000	2	0

Como se puede ver en la tabla de ejemplo la información facilitada incluye datos sobre el activo representado, la fecha y la hora de registro, los datos OHLCV y el interés inicial de cada día, este último dato no será utilizado para simplificar la complejidad y porque al ser un valor no relativo o porcentual no es tan recomendado para los modelos de aprendizaje profundo, además de que el interés puede cambiar a lo largo del día.

En un primer análisis me doy cuenta de que los datos temporales no están regularizados al formato UTC, por lo que es el primer cambio que realizo. Usando de referencia las horas de apertura y cierre de cada futuro, agrupando el volumen por horas para cada activo puedo detectar las horas con poca o nula actividad, con ello puedo inferir la zona horaria usada.

La siguiente imagen es un ejemplo de cómo lo he hecho, como se puede observar en el volumen del oro, hay una hora de descanso cada día esa es de 16:00h a 17:00h hora de Chicago, con

esto sabemos cómo ajustar a UTC. Este es uno de los pocos procesos que no he conseguido automatizar y he tenido que hacer a mano, aún así creo que podría generalizar para cada mercado, CME, etc.

Sí se generaliza por mercados -6 y +1

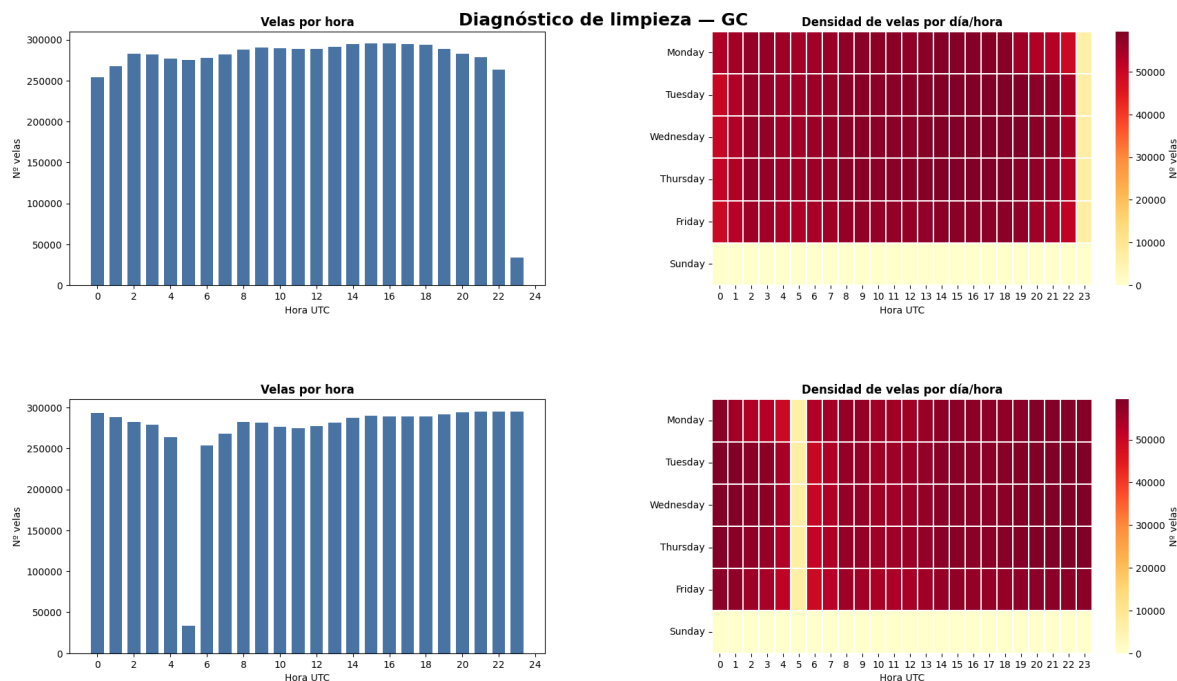


Figura 3.2: Comparación antes y después de ajustar

Explicar mejor cómo funciona la función de inferencia, decidir si pasar a UTC o no

Después de un análisis inicial, decido filtrar los datos que claramente no deberían de estar, estos son los que aparecen en los fines de semana. Los datos en horas de descanso si representan actividad real aunque sea mínima, pueden ser órdenes que llegan tarde, que se cruzan en el libro de órdenes aunque el mercado esté cerrado, actividad residual del Globex, ajustes del Exchange, ticks de apertura o cierre del libro. Además eliminarlos elimina la continuidad temporal, introduce gaps artificiales y puede afectar a diferentes métricas.

Una vez estandarizado el tiempo aprovecho la función de inferencia y añado una primera columna derivada que indica el día de *trading* aunque al ser una fecha no se usará directamente para el entrenamiento, pero sí me ayudará para calcular valores acumulados diarios, que a su vez servirán para calcular otras variables

Para mejorar el rendimiento y disminuir el tamaño de los archivos convierto los datos que

estaban en formato csv a parquet. Una ventaja extra de parquet es la habilidad de poder guardar los datos en diferentes particiones y poder leer las particiones que uno necesite, en mi caso realizo las particiones por año y por mes.

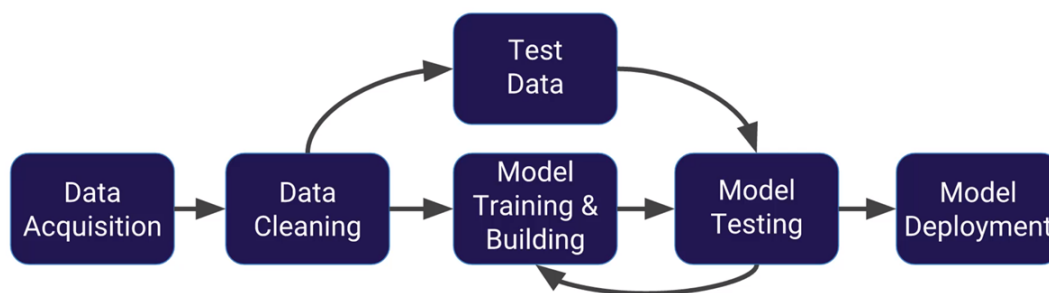


Figura 3.3: Añadir img con algunos plots de las características más usadas como modelos solos

3.2.3 Cálculo de características

Una vez con los datos OHLCV limpios, llega el momento de aplicar diferentes funciones de diversas librerías para obtener características derivadas que servirán para alimentar a los modelos.

Hay características de diferentes tipos, momento, estructural, temporal, de tendencia, de volatilidad, de volumen y características derivadas de otras. Al ser una estrategia multidía también he hecho uso de los swing charts y de características derivadas de ellas, además de otras características obtenidas de comparar diferentes activos

Para obtener estas características primero agrupo los datos que venían minuto a minuto a agrupaciones de 4 horas, de esto modo se evita tener el ruido de señales tan cortas y es un buen número para una estrategia interdía donde una posición puede estar abierta varios días

Las características están divididas en subcarpetas dependiendo de su tipo, en algunos casos existe aún más división si hay demasiadas funciones

derived

Características derivadas de otras series básicas. Incluye transformaciones estadísticas, normalizaciones, estandarizaciones y combinaciones no triviales de indicadores primarios.

momentum

Indicadores de momento y aceleración del precio. Contiene osciladores, ROC (Rate of Change) y medidas de fuerza relativa basadas en variaciones porcentuales.

returns

Cálculo de retornos simples, logarítmicos y acumulados. Incluye funciones para medir variaciones relativas entre barras y construir series de rendimiento.

selection

Módulo destinado a la selección de características. Puede incluir filtros, ranking de importancia, eliminación de colinealidad o criterios de calidad.

structural

Características que describen la estructura interna de las velas y del microcomportamiento del precio. Incluye patrones de velas, proporciones OHLC y medidas de microestructura.

swings

Detección de swings, pivotes y estructuras de máximos/mínimos relevantes. Útil para identificar cambios de tendencia y puntos de giro.

temporal

Variables temporales: hora, minuto, día de la semana, estacionalidades, ciclos intradía y cualquier codificación temporal relevante.

trend

Indicadores de tendencia: medias móviles, MACD, ADX, fuerza de tendencia y medidas de persistencia direccional.

volatility

Medidas de volatilidad histórica y realizada. Incluye ATR, bandas de Bollinger, rangos, dispersión de precios y estimadores de volatilidad intradía.

volume

Características basadas en volumen: flujo de órdenes, volumen relativo, volumen por precio, desequilibrio comprador/vendedor y estadísticas derivadas del volumen.

Una vez obtenidas todas las características se realiza una primera comprobación, donde se intenta ver si existen muchos valores nulos en las diferentes características, donde destacan la gran cantidad de valores nulos en los swings debido a la naturaleza de este dato, en este caso opto por rellenar los valores faltantes con el último valor disponible. Hay algunas características que requieren de cierto número de velas iniciales para su cálculo, estas primeras filas se puede eliminar sin problema ya que representan un porcentaje ínfimo del total de filas de datos. Debido a que algunos indicadores necesitan 12 o 14 velas para poder calcular el primer valor no me ha sido posible agrupar los datos por días ya que una de las ideas que tenía era reiniciar los indicadores todos los días, además tampoco tiene tanto sentido al tener una estrategia multidía.

En un inicio añadí solo todas las features que aparecían en el libro de [2] pero decidí añadir más para tener mayor variedad con features más complejas.

3.2.4 Selección de características

Una vez calculadas todas las características es necesario reducir ese número porque es posible que muchas de ellas estén interrelacionadas.

Para ello se aplican varias técnicas, empezando por la más básica, un filtro de alta correlación, además de eliminar las características que puedan predecir el futuro por data leakage, las no cíclicas o no estacionarias pueden confundir a los modelos indicando que un valor que quizás representaba el día de la semana sea considerado como más importante a mayor número.

También filtro usando el algoritmo de LASSO usando un modelo de aprendizaje automático clásico de sklearn que permití mayor explicabilidad, de este modo reduzco a la mitad el número de características al decidir quedarme con el 50 por ciento de características más importantes.

Todo esto ayuda a reducir el problema de la maldición de la dimensionalidad, que aunque no tiene el efecto negativo que sí tiene sobre modelos de aprendizaje automático clásicos sigue siendo útil para evitar sobreajuste y disminuye los tiempos de entrenamiento.

3.2.5 Definición y cálculo de la clase objetivo

Para este proyecto se ha decidido usar el método de la triple barrera, las tres barreras son, SL (Stop Loss), TP (Take Profit) y la barrera temporal. A partir de un límite temporal se define qué pasa primero, salta SL en el caso en el que el precio alcance un nivel de pérdida predefinido, TP es lo mismo pero con beneficio, y en caso de no llegar en el tiempo definido se considera que ha tocado la barrera del tiempo, en el caso de que en una vela se haya llegado a SL y TP nos ponemos en el peor de los casos y decidimos que ha sido SL. Los umbrales se definen por el ATR multiplicado por un valor determinado.

Aunque en un inicio se esperaba realizar una predicción de patrones de swing durante el periodo de investigación me di cuenta de que los swings tenían diferentes desventajas, partiendo por el hecho de que necesitan velas de confirmación, normalmente 2, lo que en mi caso se traduce a tener un retraso de 8 horas. Por ello he decidido optar por esta otra opción. Aún así los usaré como características para entrenar los modelos.

Además de generar una etiqueta discreta también se guarda información del retorno, en mi caso el retorno logarítmico, información que será útil para la capa de regresión, como se explicará más adelante en la próxima sección. Además, permite evaluar mejor la clasificación ya que no es lo mismo un retorno del 0.2 % que del 2 %. Se usa el retorno logarítmico porque ofrece

aditividad temporal, simetría y normalidad aproximada y permite comparaciones entre activos con volatilidades distintas.

definir el reto: lista de todos los targets posibles además de long short nada, otros de clasificación, regresión, etc y lo que conlleva cada uno

Definir los swings, y las estructuras que más se usan

3.2.6 Aprendizaje supervisado

Llegados al núcleo del proyecto, el siguiente paso consiste en definir las arquitecturas de los modelos de aprendizaje supervisado que se van a utilizar. La arquitectura se resume en un sistema de dos modelos paralelos con tres capas secuenciales y acoplados mediante un mecanismo de soft-voting que combina las predicciones de cada etapa.

Las dos primeras etapas son de clasificación binaria y la tercera de regresión. Ambos modelos están definidos usando PyTorch. Cada uno de los modelos elegidos tienen una función, LSTM está orientado a capturar dependencias a largo plazo y patrones más suaves en la secuencia, mientras que GRU tiende a reaccionar mejor a cambios rápidos, transiciones de régimen y estructuras más volátiles.

En un inicio para las dos fases de clasificación se decide realizar una votación equitativa, y si la media supera un umbral indicado para cada fase se pasa a la siguiente. Pero después decidí añadir la tercera capa para dinamizar la votación, usando el retorno esperado como ponderación, por lo que si uno de los modelos considera que va a haber mayor retorno se le da mayor peso a su predicción. El retorno se calcula a X velas.

Había pensado también en usar modelos de aprendizaje automático clásicos porque pueden captar patrones que no tienen dependencias temporales, como XGBoost y Random Forest. Pero he querido centrarme en los modelos profundos porque se adaptan mejor a las secuencias.

Tabla 3.2: Tabla comparativa de parámetros de los modelos

1	2	3	4	5
---	---	---	---	---

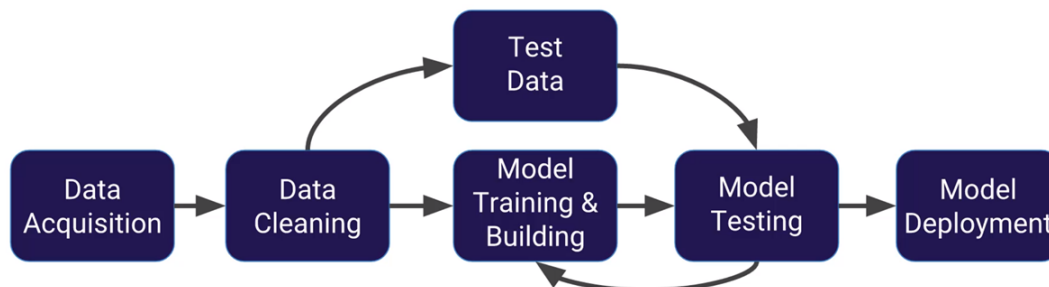


Figura 3.4: Añadir img de las capas de las redes LSTM

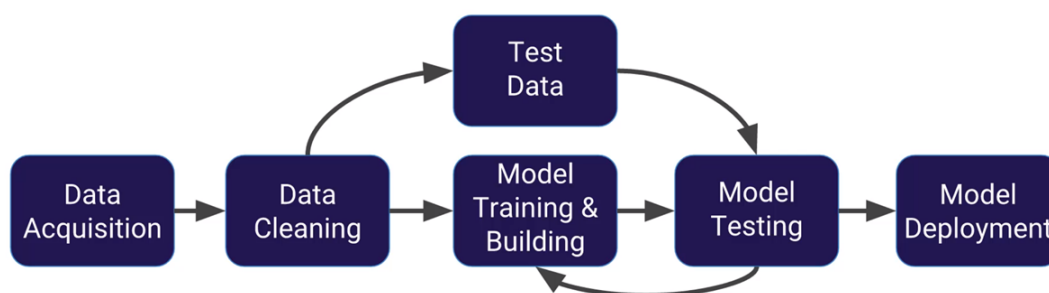


Figura 3.5: Añadir img de las capas de las redes GRU

La decisión de usar dos etapas de clasificación es debido al desbalanceo de clases, al generar el valor a predecir me doy cuenta de que aproximadamente el 70 % de las velas terminan tocando la barrera del tiempo y los otros dos se reparten 15 % cada uno, en un inicio probé modelos con salida triple pero debido al desbalance no daba muy buenos resultados incluso modificando los pesos de los valores dando mayor peso a ambas barreras no temporales no habían resultados favorables por ello decidí pasar a dos fases binarias, una primera predice si hay movimiento, si toca cualquiera de las dos barreras laterales y en caso de superar un umbral que se establece después, se pasa a la siguiente fase donde se decide la dirección del movimiento, aquí también se decide más adelante el umbral a partir del cuál se considera valido, existen dos opciones de decisión, que exista un umbral mínimo o que con que uno supere al otro por más mínima diferencia se decanta por ese, al final después de probar me decanté por añadir un umbral mínimo del 0.55.

Se me ha olvidado qué quería poner de más

Explicar el tema de los umbrales

Explicar entrenamiento

Antes de pasar al entrenamiento es necesario calcular los hiperparámetros idóneos para cada entrenamiento teniendo en cuenta el tamaño de cada dataset, el periodo de muestreo y el número de velas a futuro a predecir. Se obtienen así los siguiente hiperparámetros.

lookback

Número de velas hacia atrás que se pasa a cada momento temporal

train_size

Tamaño del bloque de entrenamiento para cada iteración

test_size

Tamaño del bloque de prueba para cada iteración

gap

Número de velas de separación entre el final de una iteración y el comienzo de la siguiente.
Sirve para evitar filtrado de datos

El entrenamiento de los modelos se hace para cada modelo por separado, entrenando las tres fases juntas, se usa un sistema de walk forward con moving window, que es la técnica requerida para entrenamiento con series temporales. A diferencia del entrenamiento normal este método respeta la estructura temporal de los datos y evita data leakage.

El walk forward (rolling forecast origin) consiste en entrenar el modelo en un bloque de inicial de datos históricos y evaluarlo en el siguiente bloque inmediatamente posterior y después se avanza hacia el siguiente bloque dejando algunas velas de margen y se repite este proceso hasta completar el entrenamiento con todos los datos posibles. La variante con moving window define cuántos datos históricos se usan en cada entrenamiento, en mi caso es una ventana de tamaño fija, existe otra opción llamada expanding window donde la ventana va aumentando de tamaño.

En las series temporales no sirve el cross validation clásico porque mezcla aleatoriamente los datos, divide los folds sin respetar el orden temporal, esto provoca que el modelo pueda entrenar con datos del futuro introudciendo data leakage masivo.

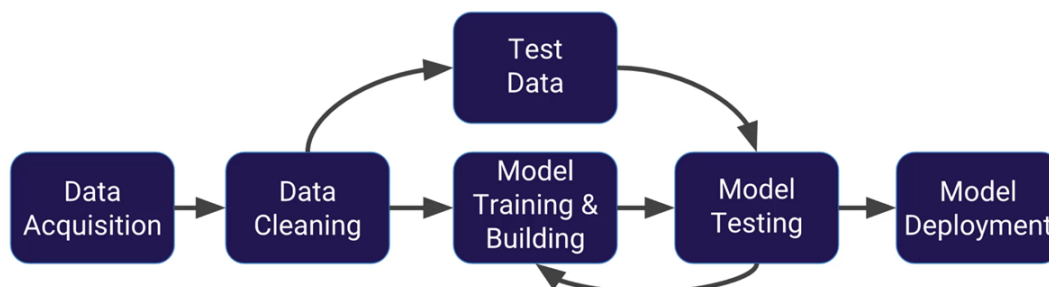


Figura 3.6: Añadir img de walk forward

Una vez explicado el método a utilizar y después de obtener estos datos pasamos a crear los dataset que se van a usar en cada época teniendo en cuenta el lookback calculado en el paso anterior.

Pasamos al entrenamiento en sí, como se mencionaba antes, entrenamos todas las capas a la vez pero de manera independiente. Lo primero que se hace es para cada etapa separar los datos en train y test, acto seguido se usa un scaler para escalar los datos de entrenamiento, se hace en este orden para evitar leakage que ocurriría si se aplicase el scaler directamente a train y test. Se entrena primero L1 y después L2 y por último L3. El resultado del entrenamiento es un dataset con la predicción de cada capa y la confianza que tiene en dicha predicción.

Después de entrenar se decide primero buscar los umbrales de confianza adecuados para las dos primeras capas, ya que la tercera al ser una capa de regresión no cuenta con una confianza. Se usa la métrica de aprendizaje automático de f1 macro ya que no se ve afectada por el desbalance de clases como sí ocurre con accuray, precision o recall.

Se guardan los modelos en archivos pth, junto al último scaler, ya que el scaler se actualiza en cada ventana, y por lo tanto sus parámetros cambian (media, desviación estándar, etc). Además, se guardan las features usadas para poder filtrar después durante el backtest y la operación, y los parámetros del modelo usado, como el umbral de confianza.

El scaler que he decidido usar es el RobustScaler, ya que ofrece varias ventajas, usa la mediana y el rango intercuartil (IQR) para escalar los datos, lo que lo hace resistente a valores atípicos conocidos como cisnes negros, y a cambios bruscos del mercado. Existen otras opciones como StandarScaler y MinMaxScaler, el primero produce escalados inestables y el segundo usa un rango fijo.

La votación solo ocurre si ambos modelos pasan a la segunda fase y predicen la misma dirección si no no se hace nada.

Una de mis ideas es que a futuro se puedan ir actualizando tanto el modelo como el scaler de

manera semanal, pero esto se explicará más adelante. La idea es poder adaptarse mejor a los cambios de régimen del mercado.

Faltan cosas por explicar

Anteriormente se ha mencionado el uso de modelos clásicos para permitir mayor explicabilidad, en deep learning existe la opción de usar Shap DeepExplainer para ver el peso de las diferentes características en la predicción de un modelo pero aunque sea una herramienta potente para evitar que las redes neuronales sean cajas negras, Shap asume que las características son independientes entre sí. Además, Shap es costoso y proporciona explicaciones locales, por cada predicción, por lo que no necesariamente da una visión global.

3.2.7 Aprendizaje no supervisado

Además de usar modelos de aprendizaje supervisado tenía la idea de usar modelos no supervisados para varias tareas.

En primer lugar, para agrupar los diferentes activos en diferentes grupos. En vez de realizar agrupaciones arbitrarias, por ejemplo, todas las monedas juntas, dejo que el sistema decida cuales son los activos más parecidos. De este modo podría generar más características derivadas de la comparación de los activos, basandome en el concepto de *Pairs Trading* donde se comparan un par de activos que se saben que están correlados y cointegrados, y si uno de ellos sube más que el otro se abre una posición en corto y si uno de ellos baja más que el otro se abre una posición en largo.

El problema que le encuentro a este primer caso de uso es la disparidad de datos, por una parte, los datos históricos proporcionados no son de las mismas fechas, esto se soluciona fácilmente al recortar los datos más antiguos ya que además podrían causar problemas a los modelos. Pero donde existen más problemas es el tema de los horarios de las diferentes bolsas, la idea sería usar la función de rellenado hacia adelante para los momentos donde faltasen datos de alguno de los futuros. Otra opción sería comparar cada futuro con el centroide (el centro de cada cluster) del cluster al que pertenece cada futuro y el resto de centroides, de este modo se puede evitar problemas por la disparidad del número de futuros en cada cluster.

Además de este problema está la necesidad de tomar la decisión de qué tipo de modelo no supervisado usar, si uno donde se predefine el número de clusters o uno donde el propio modelo decide el número (Decidir entre modelos paramétricos (k fijo) o modelos no paramétricos (k variable))

Y el problema principal sería sobre qué agrupar qué características usar para la agrupación.

El segundo uso es el de segmentar los regímenes de mercado. Con esta información hay dos

opciones de acción, por una parte, usarlo como una característica más para complementar al resto, aunque puede no ser tan útil al usar modelos de aprendizaje supervisado profundo que al estar diseñados para manejar series temporales pueden inferir los regímenes de mercado por si solos. La segunda opción es usar la diferencia de regímenes para entrenar modelos dedicados para cada régimen consiguiendo modelos más especializados en cada régimen. Este uso tiene los mismos problemas que el primero.

3.2.8 Análisis de resultados de entrenamiento

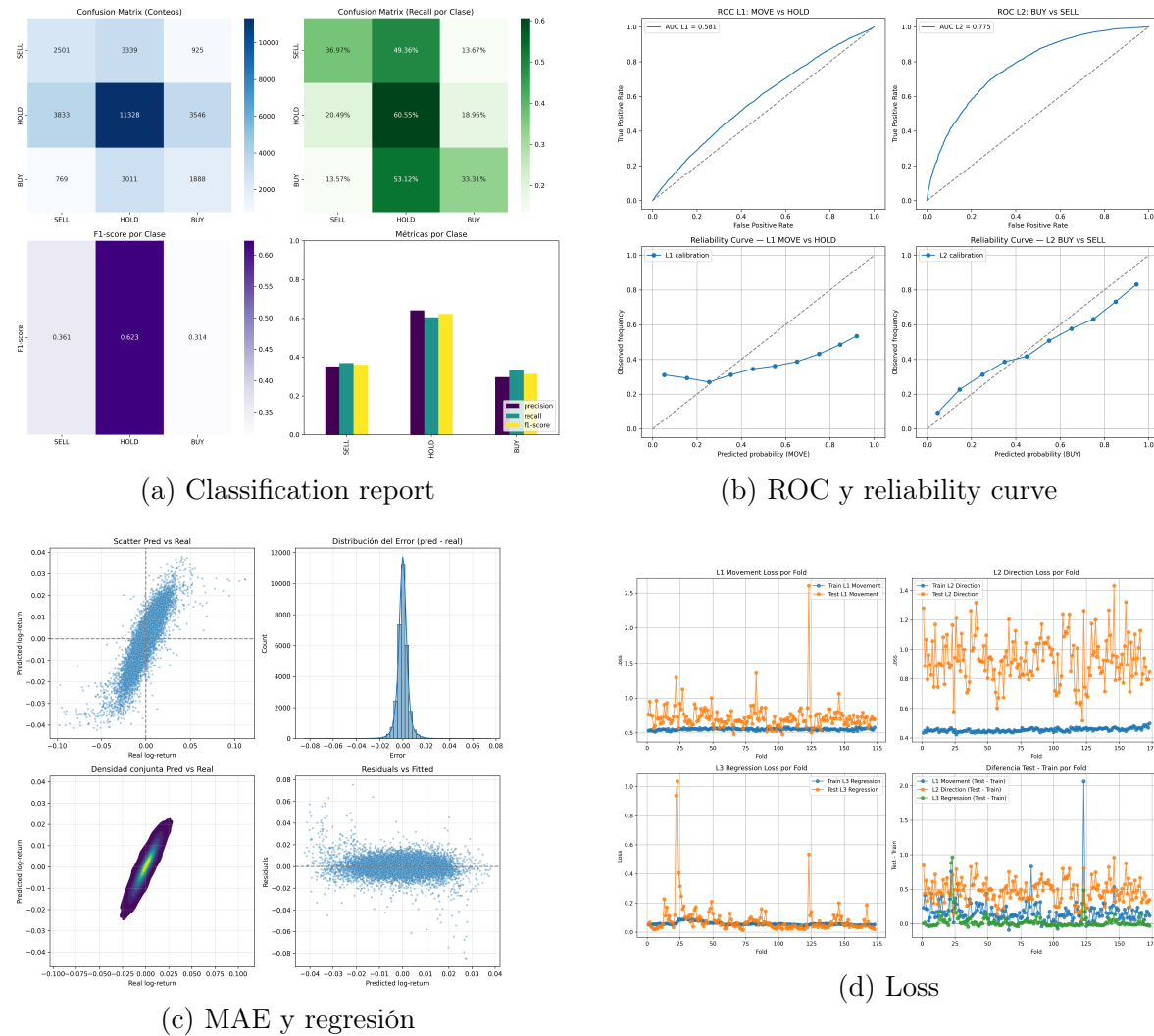
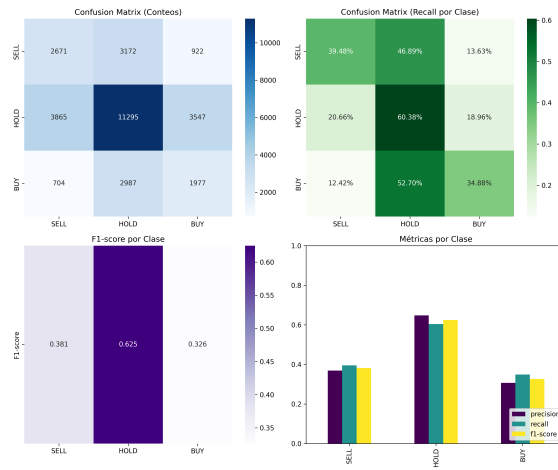
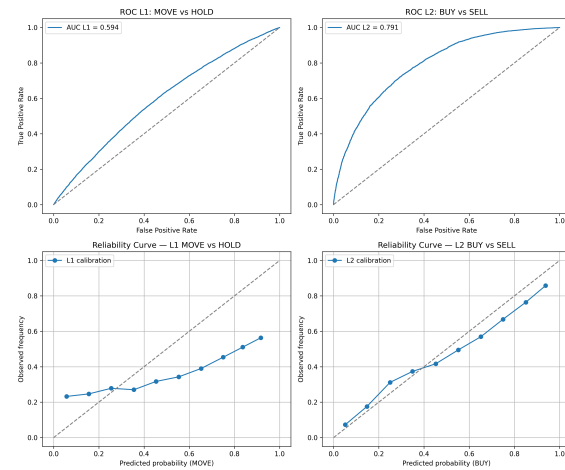


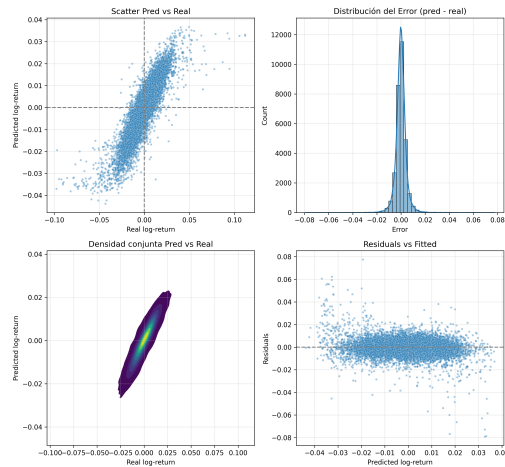
Figura 3.7: Resultados obtenidos por el modelo LSTM.



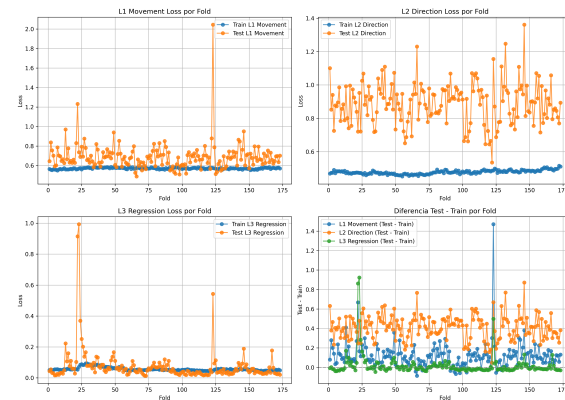
(a) Classification report



(b) ROC y reliability curve



(c) MAE y regresión



(d) Loss

Figura 3.8: Resultados obtenidos por el modelo GRU.

Añadir curva auc roc para las dos capas de clasificación

Añadir curva loss por fold

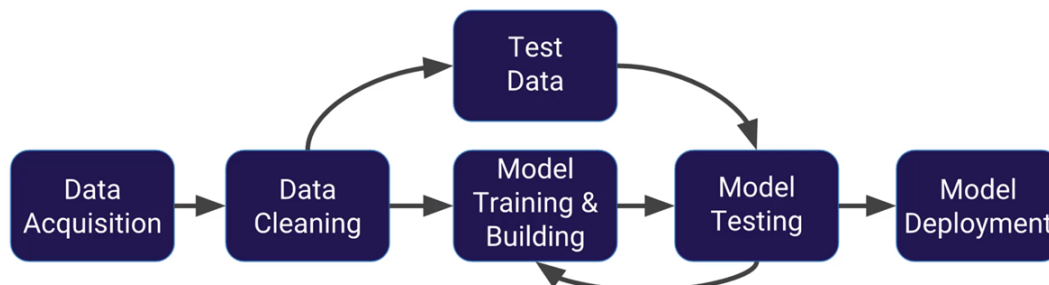


Figura 3.9: Añadir img con evolución de las métricas dependiendo de los umbrales

Tras el entrenamiento de los modelos es necesario analizar qué tan buenos son usando métricas tanto de clasificación como de regresión.

En el primero caso, para clasificación, existe el classification report que nos da un conjunto de las métricas más populares. Para regresión calculamos las métricas.

3.2.9 Backtest de las predicciones

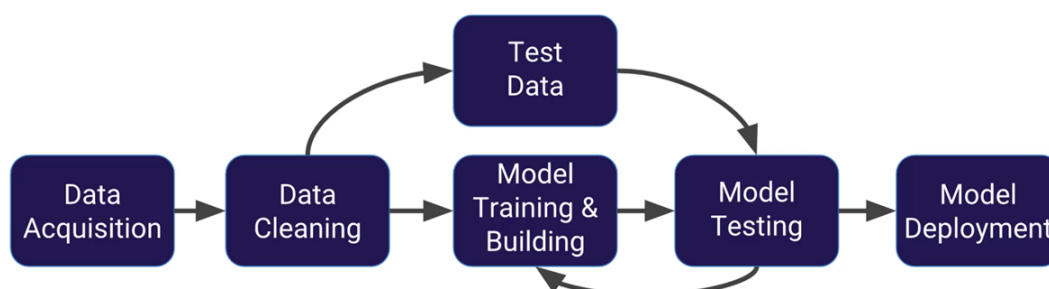


Figura 3.10: Añadir img con una simulación del Backtest

Después de comprobar que nos convencen los resultados pasamos al backtest, la idea es tener unas condiciones lo más parecido a simular una operación real con los modelos solo que carece de algunas desventajas como la latencia o el spread. Aún así es un muy punto de partida para ver qué tal se deselvuelve el modelo. En este caso las métricas a usar son Max Drawdown, Sharpe Ratio y Alpha

Ver si se puede meter un contador con racha de victorias y derrotas

Max Drawdown

Sharpe Ratio

Alpha

El backtest se hace con datos con intervalo de 30 mins porque la API tiene limitaciones de timeout

Se han probado cuatro estrategias de operación

- Una única posición abierta como máximo y de tamaño 1
- Se pueden varias posiciones seguidas hasta 5 pero sigue siendo de tamaño 1
- Una única posición abierta como máximo y de tamaño variable hasta 5
- Se pueden varias posiciones seguidas hasta 25 y de tamaño variable hasta 5

	Scale-in: No	Scale-in: Sí
Sizing dinámico: No	Modalidad 1 1 posición tamaño fijo (1)	Modalidad 2 hasta 5 posiciones tamaño fijo (1)
Sizing dinámico: Sí	Modalidad 3 1 posición tamaño variable (≤ 5)	Modalidad 4 hasta 25 posiciones tamaño variable (≤ 5)

Tabla 3.3: Cuadrícula de modalidades operativas según uso de scale-in y sizing dinámico.

Hay dos fases en el Backtest, primero se decide qué considerar buena predicción, hay 4 casos

	Long y Short	Solo Long
Todas las predicciones	Modalidad 1 todas las predicciones que han superado los umbrales definidos en el entrenamiento	Modalidad 2 hasta 5 posiciones tamaño fijo (1)
Predicciones que superar un umbral más estricto	Modalidad 3 1 posición tamaño variable (≤ 5)	Modalidad 4 hasta 25 posiciones tamaño variable (≤ 5)

Tabla 3.4: Cuadrícula de modalidades operativas según uso de scale-in y sizing dinámico.

3.2.10 Simulación de Monte Carlo

Como paso siguiente al backtest pasamos a una simulación de Monte Carlo sencilla. Existen dos posibilidades, reordenar las predicciones ya hechas o intentar generar nuevos datos OHLCV con los que alimentar los modelos, los datos se generarían a partir de medias y varianzas.

Para este proyecto he preferido quedarme con la versión simple porque se puede obtener una aproximación fiable de la dispersión de las métricas sin la necesidad de generar nuevos datos.

La versión de Monte Carlo utilizada en este proyecto se basa en un block bootstrap con solapamiento, consiste en remuestrear por bloques en vez de por valores sueltos para mantener parte de la relación temporal, y tiene solapamiento para garantizar que todos los posibles segmentos de la serie original puedan aparecer en la simulación y no solo bloques fijos de x observaciones. En mi caso uso bloques de 24 velas, aproximadamente una semana de predicciones.

Aunque la estrategia es interdiaria, al realizar un backtest de como mucho 5 meses se ha decidido agrupar los PnL por día ya que si se hacía por semanas para algunos activos solo habría de 2 a 4 PnL mientras que si se agrupa por días en el peor de los casos habrían mínimo más de una decena de días con actividad

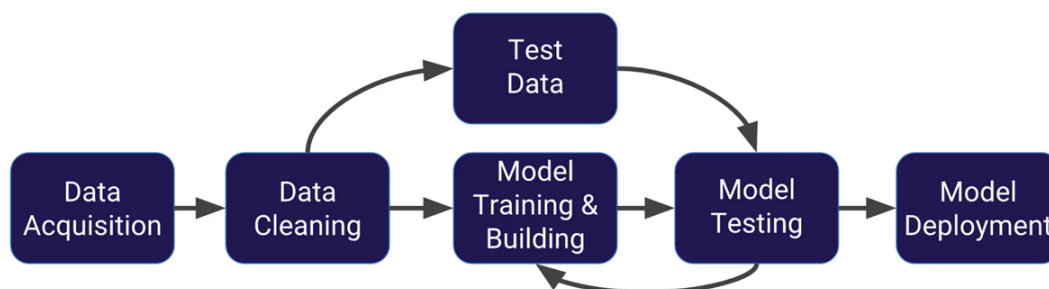


Figura 3.11: Añadir img con una simulación de Monte Carlo con los percentiles

3.2.11 Gestión de cartera

Una vez entrenados los modelos para cada futuro es necesario coordinarlos todos para que no hayan problemas de liquidez. Para ello en un primer momento decidí usar un método más rudimentario y manual para después pasar a la optimización de la cartera usando Pyomo

En el primer caso se realizan los siguientes pasos

1. Realizar todas las predicciones para todos los futuros disponibles en el momento temporal

2. Ordenar por nivel de confianza total todos los que hayan pasado de la primera capa
3. Iterar sobre cada predicción por este orden e ir comprobando si cumple con los requisitos para poder operar como la disponibilidad de capital suficiente y las restricciones como el Value at Risk, exposición máxima
4. Ejecutar las diferentes operaciones

Una vez que tuve el sistema funcionando paso a implementar un optimizador usando Pyomo, para solventar algunas limitaciones que tiene el método manual, principalmente el hecho de que una predicción con menor confianza pueda generar mayor beneficio.

Explicar qué es Pyomo

Para ello sigo los siguiente paso para una optimización de un objetivo, en este caso maximizar el *Sharpe Ratio*

1. Definir variables, conjunto de activos,
2. Definir parámetros, retorno esperado, volatilidad, matriz de covarianza, capital disponible, límite de exposición, restricciones de riesgo
3. Definir variable de decisión, abrir o no una posición
4. Definir restricciones, suma de pesos, límite de exposición, restricciones de riesgo, restricciones operativas
5. Definir función opjetivo, Sharpe Ratio
6. Seleccionar solver
7. Interpretar solución

Implementar la gestión de cartera

Tabla 3.5: Tabla comparativa de gestión de riesgo

1	2	3	4	5
---	---	---	---	---

3.2.12 Análisis de resultados de optimización y Backtest de las optimizaciones y Simulación de Monte Carlo

Comparar respecto a usar todo el capital en un solo activo

Pasamos a realizar otro backtest, además de comparar con la operación de un solo activo.

Realizamos otra simulación de Monte Carlo

Para realizar el backtest y la simulación de Monte Carlo se ha hecho lo siguiente, se cargan todos los trades de todos los futuros usando la estrategia óptima obtenida anteriormente, se crea un DataFrame y se agrupan por fechas, se simula usando una cuenta con 1M para poder simular también los límites de margen inicial y de mantenimiento. Se ordenan las predicciones por confianza total y se comprueba que no se exceden los márgenes.

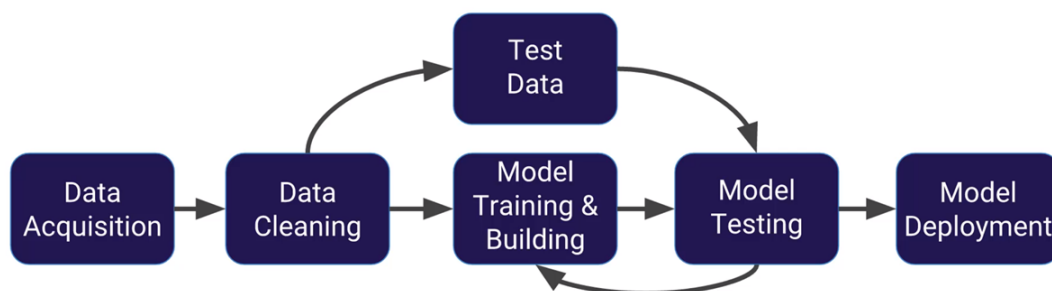


Figura 3.12: Añadir img con Backtest

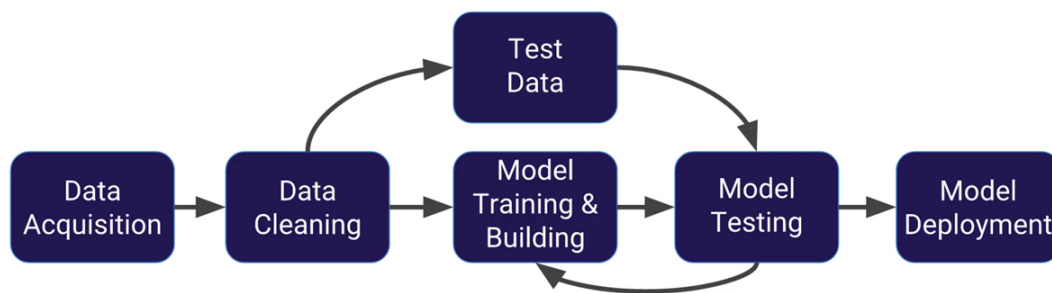


Figura 3.13: Añadir img con una simulación de Monte Carlo

3.2.13 Manejo del riesgo

Una vez entrenados los modelos y antes de pasar a la operación es necesario definir el riesgo que uno está dispuesto a tener. La métrica más importante en este sentido es el *Max Drawdown*, que define la mayor caída del valor neto de la cartera que estamos dispuestos a aceptar, si se llega a ese límite, se cierran todas las posiciones del resto de activos. En mi caso lo defino como un valor porcentual. Relacionado con esto está el porcentaje máximo de activo en posiciones abiertas que estoy dispuesto a mantener simultáneamente, es decir, el porcentaje máximo del equity expuesto. Esta métrica define el límite superior de capital que puede estar comprometido en operaciones abiertas en un momento dado, independientemente del apalancamiento implícito de cada instrumento *Exposure Limit*.

Por otra parte a la hora de operar hay que tener en cuenta que el spread al inicio y al final de las sesiones puede ser mayor que durante el resto del tiempo, por ello al tener información de las horas de mercado es posible decidir que para esos instantes temporales en vez de operar a en punto, decido atrasar la toma de decisiones a 15 minutos después o una vez que se supere un umbral de volumen en la apertura y adelantar 15 minutos o más si el volumen disminuye. El volumen puede servir en todas las horas para evitar slippage.

Por último como medida para estar al día con los datos de los futuros, cada semana se entrenan nuevos modelos o se afinan los modelos actuales y se comparan con los modelos actuales para comprobar cuales son mejores predictores de los datos de la última semana o mes. Esto se explicará en más detalle en la sección de orquestación.

En el caso de haber seguido con el clustering usando aprendizaje no supervisado también se podría indicar un umbral máximo de exposición por grupo, evitando así que en el caso de que las señales positivas pertenezcan todas a un único grupo y este haya sido predicho incorrectamente perder demasiado capital.

También se realizan comprobación para la integridad de los datos, por ejemplo que Open y Close estén entre Low y High, que Low sea menor o igual a High, volumen no negativo, que no hayan duplicados.

3.2.14 Operación

Una vez con todo preparado llega el momento de unir todo en un único código que será el que se conecte a la API de Interactive Brokers para realizar las operaciones.

La operación tendrá los siguientes pasos de manera resumida

1. Conexión a la API de Interactive Brokers

2. Lectura de los futuros sobre los que se van a operar y obtener los contratos específicos a operar
3. Descarga de los datos históricos necesarios para realizar las predicciones iniciales
4. Carga de la información de configuración del sistema
5. Carga de los modelos, los escaladores, y la configuración individual de cada futuro
6. Cada X minutos realizar las predicciones de cada futuro
7. Tener watchdog de vigilancia para control del riesgo

Como los futuros tienen fecha de caducidad obtengo una lista para cada futuro con el número de contratos activos y los ordeno para obtener el contrato con mayor volumen, minimizando el spread. En el caso de estar corriendo el código durante varios días seguidos el sistema realiza esta consulta una vez al día. En caso de que sigan habiendo contratos abiertos existen dos posibilidades dependiendo de cómo esté configurado, en el primer caso cierra esos contratos directamente, en el segundo caso comprueba la diferencia de volumen y si es menor de un umbral decide seguir con el contrato anterior aunque las predicciones futuras ya se realizan sobre el contrato con mayor volumen, si se supera el umbral, hace como en el primer caso y cierra las posiciones.

Existe la posibilidad de elegir el comportamiento a la hora de comprar un activo, la opción más sencilla es que si sale que hay movimiento se hace long o short abriendo una sola posición, podría usarse el log return o la volatilidad para decidir abrir más posiciones siempre respetando los límites de capital impuestos por el broker y por uno mismo. Otra opción es a la hora de una segunda y sucesivas predicciones, como se ha mencionado antes se puede decidir no hacer nada en caso de seguir con la misma predicción pero también se puede decidir añadir más posiciones ya sea usando el mismo umbral de confianza o pidiendo un umbral mayor, y que en este caso si no supera el nuevo umbral pero si el original mantener la posición. En el caso de que la señal sea la opuesta que la original se cierra la posición anteriormente abierta y se abre una posición opuesta, en el caso de una predicción neutra se puede elegir si cerrar o dejar que siga hasta llegar a alguna de las tres barreras o a alguna predicción diferente, todo esto se configura en un json.

En otro json se guardan todas las operaciones que se realizan

```
{
  "id": 1248,
  "symbol": "GC",
  "status": "closed",
  "side": "long",
  "quantity": 1,
```

```

    "timestamp_open": "2024-03-12T10:40:00Z",
    "timestamp_close": "2024-03-13T14:00:00Z",
    "entry_price": 2145.3,
    "exit_price": 2162.7,
    "stop_loss": 2138.0,
    "take_profit": 2170.0,
    "pnl": 174.0,
    "pnl_pct": 0.81,
    "return": 0.0081,
    "max_favorable_excursion": 0.0124,
    "max_adverse_excursion": -0.0048,
    "model_signal": "BUY",
    "model_confidence_l1": 0.87,
    "model_confidence_l2": 0.72,
    "model_reg_value": 0.0045,
    "thresholds_used": {
        "threshold_move": 0.55,
        "threshold_dir": 0.60
    },
    "commission": 2.5,
    "slippage": 0.0,
    "equity_before": 10234.5,
    "equity_after": 10408.5,
    "bar_index_open": 58291,
    "bar_index_close": 58347,
    "durationBars": 56,
    "duration_minutes": 1344
}

```

3.2.15 Logging

No es una fase per se, es algo que se debe implementar en las diferentes etapas del flujo para saber qué está pasando en cada parte y para poder llevar un registro de todo.

Por ejemplo, en la fase de entrenamiento se pueden mostrar mensajes sobre qué futuros se están entrenando y los resultados que se están obteniendo, lo mismo con los backtest y simulaciones.

El punto más importante donde tener logs es durante la operación, donde se muestran información como el número de posiciones abiertas, el estado de conexión con la API, qué contrato se ha seleccionado, si se han descargado bien los datos, si se han cargado los datos, información de las configuraciones cargadas, las predicciones realizadas, las órdenes de ejecución, el watchdog.

3.3 Despliegue usando contenedores de Docker

Una vez que se ha desarrollado el autómata y se ha comprobado que su funcionamiento es correcto, he decidido añadir el proyecto a un contenedor de docker para facilitar el despliegue en otros equipos

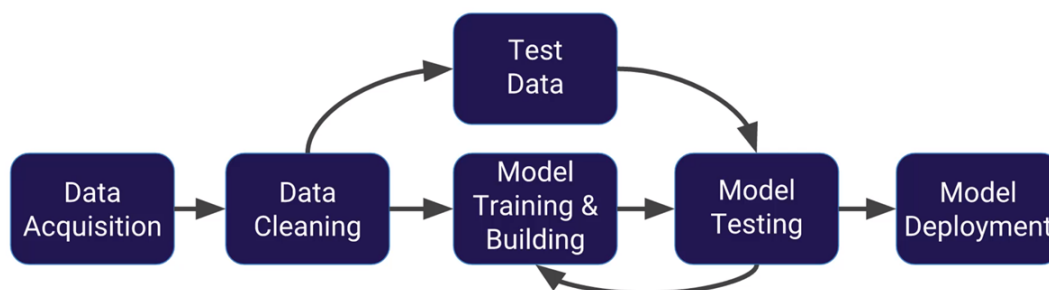


Figura 3.14: Añadir img con el diagrama de cómo funciona todo, los puertos usados, etc

3.3.1 Configuración del contenedor principal

Por cómo he desarrollado el sistema y por velocidad todo va a estar incluido en un único contenedor.

No requiere demasiados cambios ya que voy a copiar los códigos desarrollados a dentro del contenedor.

De este modo se logra tener un entorno estático donde se han seleccionado todas las versiones de los diferentes paquetes usados en el proyecto.

Además, esto permite agrupar todas las funcionalidades que quiero implementar en un mismo lugar, en lugar de tener diferentes contenedores con funciones dedicadas.

El contenedor ejecuta el código de operación que se encarga de todo.

Solo es necesario tener P5Operation.py en el contenedor y el paquete desarrollado para poder acceder a todas las funciones necesarias. El entrenamiento se puede hacer directamente desde local.

3.3.2 Bases de datos

Al tener el proyecto dentro de un contenedor veo lógico tener una base de datos con diferentes tablas para guardar diferentes métricas.

Uso una base de datos relacional para guardar métricas de la operación y el historial de operaciones, además de guardarlo como archivos json como lo hacia antes. En este caso uso PostgreSQL y para poder visualizar con mayor facilidad los datos uso PgAdmin aunque también se puede ver parte de la información desde grafana, contenedor que se explica en el siguiente apartado.

Como he mencionado anteriormente voy a guardar varios tipos de datos, por ello necesito diferentes tablas

Detallar las tablas

Tabla 3.6: Estructura de la tabla `metrics`

Campo	Tipo	Descripción
id	SERIAL	Identificador único
ts	TIMESTAMPTZ	Marca temporal de la métrica
metric_name	TEXT	Nombre de la métrica (pnl, sharpe, latency, etc.)
value	DOUBLE PRECISION	Valor numérico de la métrica
tags	JSONB	Información adicional (símbolo, timeframe, etc.)

Tabla 3.7: Estructura de la tabla `risk_events`

Campo	Tipo	Descripción
id	SERIAL	Identificador único
ts	TIMESTAMPTZ	Momento del evento
event_type	TEXT	Tipo de evento (drawdown, latency, stale_data...)
severity	TEXT	Nivel (info, warn, critical)
details	JSONB	Información adicional del evento

Tabla 3.8: Estructura de la tabla `signals`

Campo	Tipo	Descripción
id	SERIAL	Identificador único

Continúa en la siguiente página

Campo	Tipo	Descripción
ts	TIMESTAMPZ	Marca temporal de la señal
symbol	TEXT	Símbolo del futuro
contract	TEXT	Contrato específico
signal_value	DOUBLE PRECISION	Valor de la señal
model_version	TEXT	Versión del modelo
features	JSONB	Features usadas en la predicción
decision	TEXT	accepted / rejected
reason	TEXT	Motivo del rechazo (spread, volumen, riesgo...)
tags	JSONB	Información adicional

Tabla 3.9: Estructura de la tabla **trades**

Campo	Tipo	Descripción
id	SERIAL	Identificador único
symbol	TEXT	Símbolo del futuro
contract	TEXT	Contrato específico
status	TEXT	Estado de la operación
side	TEXT	Dirección
quantity	INTEGER	Número de contratos
timestamp_open	TIMESTAMPZ	Momento de apertura
timestamp_close	TIMESTAMPZ	Momento de cierre
entry_price	DOUBLE PRECISION	Precio de entrada
exit_price	DOUBLE PRECISION	Precio de salida
stop_loss	DOUBLE PRECISION	Stop loss asignado
take_profit	DOUBLE PRECISION	Take profit asignado
pnl	DOUBLE PRECISION	Beneficio/pérdida absoluto
pnl_pct	DOUBLE PRECISION	Beneficio/pérdida porcentual
return	DOUBLE PRECISION	Retorno relativo
max_favorable_excursion	DOUBLE PRECISION	Máximo movimiento a favor
max_adverse_excursion	DOUBLE PRECISION	Máximo movimiento en contra
model_signal	TEXT	Señal generada por el modelo
model_confidence_l1	DOUBLE PRECISION	Confianza del modelo (nivel 1)
model_confidence_l2	DOUBLE PRECISION	Confianza del modelo (nivel 2)
model_reg_value	DOUBLE PRECISION	Valor regresivo del modelo
thresholds_used	JSONB	Umbral usados en la decisión
commission	DOUBLE PRECISION	Comisión aplicada
slippage	DOUBLE PRECISION	Slippage registrado
equity_before	DOUBLE PRECISION	Equity antes de la operación

Continúa en la siguiente página

Campo	Tipo	Descripción
equity_after	DOUBLE PRECISION	Equity después de la operación
bar_index_open	INTEGER	Índice de barra de apertura
bar_index_close	INTEGER	Índice de barra de cierre
duration_bars	INTEGER	Duración en barras
duration_minutes	INTEGER	Duración en minutos

3.3.3 Visualización

Por último, una vez que todo funciona en contenedores he decidido añadir un sistema de visualización de toda la información que se recopila en directo y en diferido.

Para ello uso varios contenedores, pushgateway, prometheus, loki, promtail, y grafana.

Prometheus y Pushgateway gestionan métricas; Promtail y Loki gestionan logs. Pushgateway y Promtail son los recolectores, mientras que Prometheus y Loki son los motores de almacenamiento y consulta. Por parte de SQL, Grafana tiene conexión directa con el contenedor de Docker que lo alberga.

Toda la información se consume después en un *dashboard* en grafana donde se agrupa toda la información.

En la visualización se observan diferentes métricas empezando por las relacionadas con el rendimiento del sistema, Equity curve, max drawdown, sharpe ratio, volatilidad diaria, profit factor, win rate, rendimiento por futuro. Por otra parte están

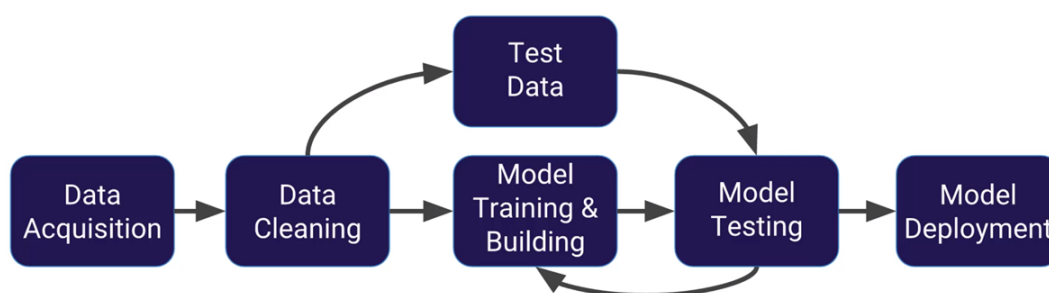


Figura 3.15: Añadir img dashboard grafana

3.3.4 Alertas y avisos

Además de grafana consideraba buena idea tener un sistema de alerta para recibir notificaciones de grandes eventos, estos se definen por reglas en Alertmanager. Cuando salta alguna de las reglas el sistema envía una notificación al móvil a través de Slack que se ha configurado usando una API key.

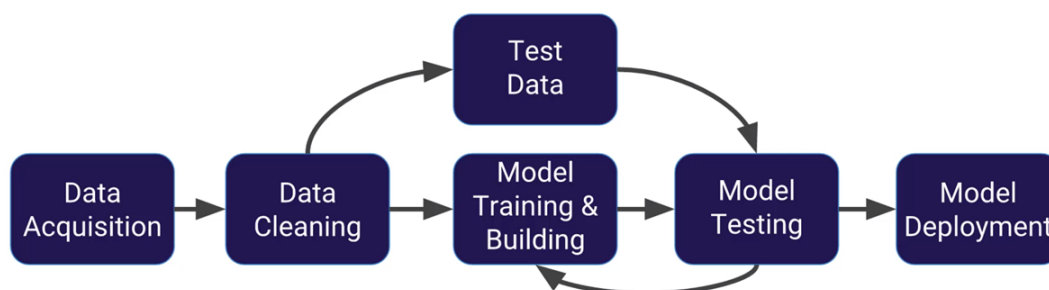


Figura 3.16: Añadir img Alertmanager o Slack

3.3.5 Orquestación y MLOps

Tanto Airflow como MLflow, ver si hay tiempo para implementarlos

Una vez desplegado un sistema robusto que cuenta con un autómata funcional y con alertas tanto de seguridad como para mostrar información, decido implementar el uso de Airflow como orquestador.

En general el uso de Airflow está muy ligado al uso de MLflow ya que será el uso principal de esta herramienta por mi parte.

En MLflow

Apache Airflow es un orquestador de flujos de trabajo que permite programar, monitorizar y ejecutar tareas complejas de forma secuencial o paralela mediante DAGs (Directed Acyclic Graphs).

MLflow es una plataforma para gestionar el ciclo de vida de modelos de Machine Learning: tracking, versionado, artefactos, modelos, comparaciones y despliegue.

Los usos principales de MLflow para mí son el registro de experimentos, que en vez de realizar los entrenamientos manualmente se realicen aquí permitiendo guardar toda la información de cada entrenamiento, permite comparar diferentes modelos y de este modo decidir si es hora de

cambiar un modelo por una versión actualizada o un nuevo modelo, permite llevar un registro de los modelos que se están utilizando, de los usados o los que están probándose. Facilita la búsqueda de parámetros de los modelos.

Airflow por su parte sirve para orquestar muchas de las funcionalidades de MLflow, orquestación del ciclo de vida del modelo, entrenamientos semanales, fine tuning incremental con los datos, evaluación automática de nuevos modelos, comparación contra el modelo actual, promoción del mejor modelo a producción, también permite automatización del bot, que despliegue el bot los sábados por la tarde y lo pare el viernes por la tarde, reinicios controlados.

El problema principal es que Airflow es muy pesado y consume mucha RAM por lo que algunas de las funcionalidades no se pueden realizar aún así se podría activar manualmente los fines de semana para poder realizar las tareas relacionadas con el ciclo de vida de los modelos.

Resultados

4.1 Resultados

Como se han mencionado en capítulos anteriores el resultado de las pruebas ha sido positivo

En la gráfica siguiente se muestra cómo va evolucionando el capital de la cuenta simulada, se puede observar todas las predicciones que realizan los modelos y el sharpe ratio final.

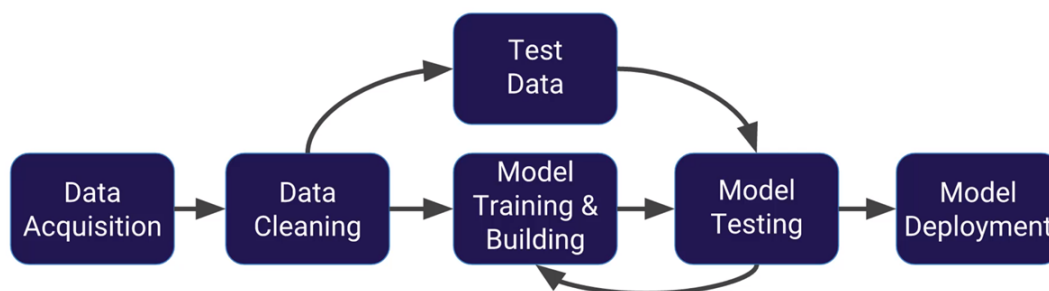


Figura 4.1: Añadir img con equity curve, max drawdown y sharpe ratio

Para poner en perspectiva los resultados he decidido compararlos con los obtenidos por los participantes de la competición Robotrader, competición en la cuál iba a participar en un inicio pero que por incompatibilidad con el desarrollo de este proyecto no acabé participando. Como se puede observar la curva mejora a muchos de los sistemas presentados.

La competición se realizó durante los meses de marzo a mayo, estos datos nunca se han usado para entrenar los modelos para evitar leakage. No es una comparación perfecta ya que no se tienen en cuenta todos los problemas que pueden haber en vivo pero se han intentado simular lo mejor posible.

Además de mostrar los datos teóricos que se hubiesen conseguido también he querido comparar con los datos de la simulación de Monte Carlo.

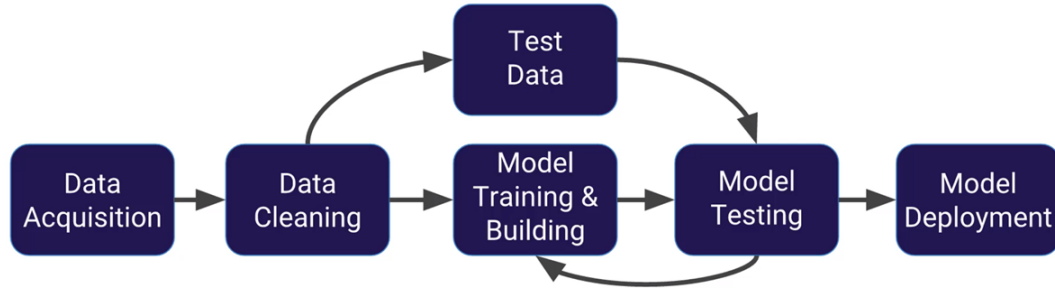


Figura 4.2: Añadir img comparativa del Backtest vs Robotrader

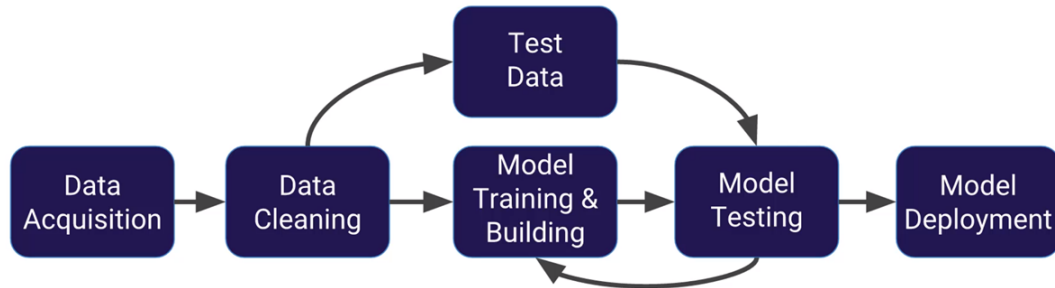
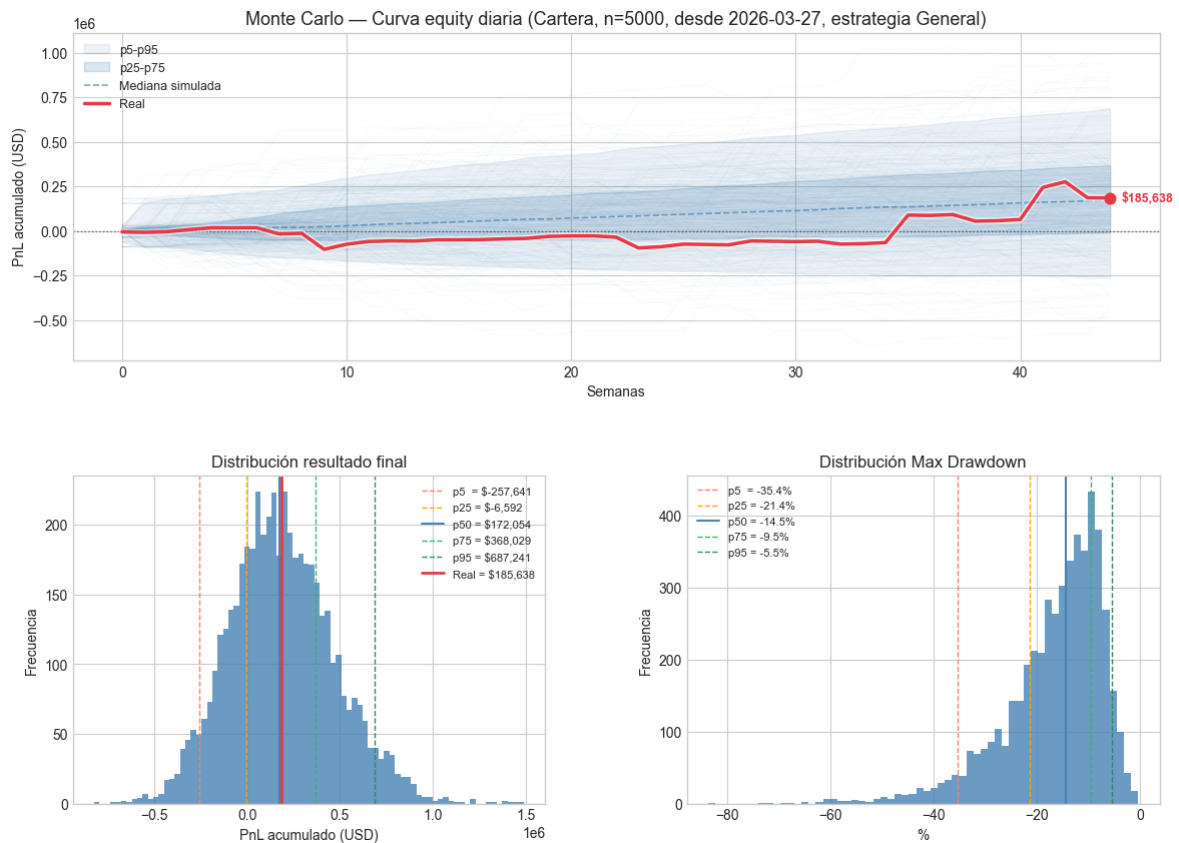


Figura 4.3: Añadir img comparativa del Monte Carlo vs Robotrader



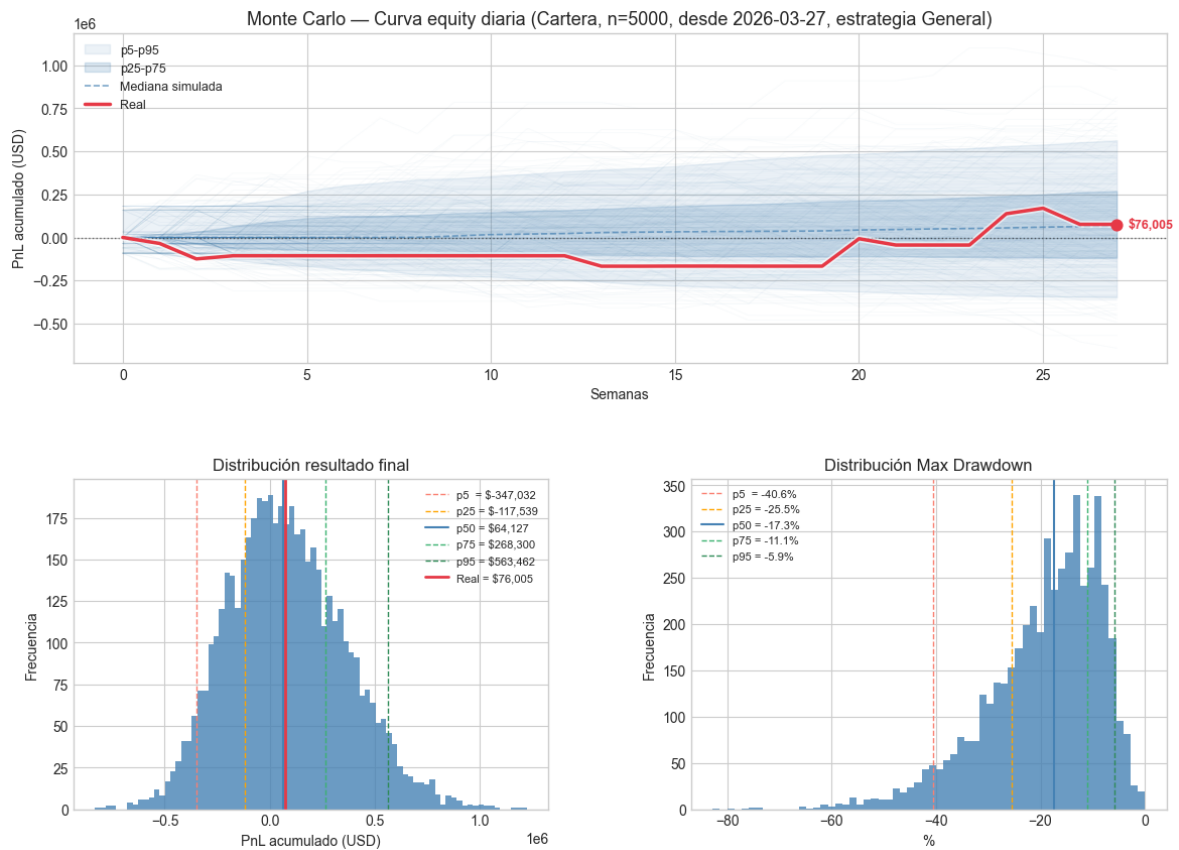


Figura 4.5: Añadir img comparativa del Monte Carlo vs Robotrader

Añadir grafica de la evolución del equity, comparativa contra Robotrader

Añadir Monte Carlo

Futuro	Capital inicial	Capital final				Ganancias mejor caso
		Normal	Scale-in	Sizing	Scale-in + Sizing	
AD	30k	No hay operaciones				0
BP	30k	Por alguna razón faltan dos features				0
CL	300k	305k	317k	306k	400k	100k
EC	40k	No hay operaciones				0
ES	350k	320k	280k	349k	250k	-1k
GC	450k	450k	465k	450k	465k	15k
MFXI	5k	4.8k	4.75k	4.5k	3.75k	-0.2k
NG	70k	70k	69k	71k	67k	1k
NQ	650k	630k	570k	650k	550k	0
YM	200k	198k	198k	200k	196k	0
ZB	50k	50k	49k	50k	48k	0
ZN	30k	25k	24k	22k	0k	-5k
ZS	50k	300k	300k	80k	300k	250k

Tabla 4.1: Tabla resumen de los 13 futuros evaluados en las cuatro modalidades operativas. Desde 2026-01-20 hasta 2026-06-01

Como capital inicial he usado 10 veces el margin inicial mínimo para abrir una posición

Futuro	Capital inicial	Ganancias mejor caso	Max DD	Ratio
CL	300k	0		
ES	350k	-1k		
GC	450k	8510	6730	1.26
MFXI	5k	-0.2k		
NG	70k	904	2116.23	0.43
NQ	650k	17385	0	No aplica
YM	200k	6825	2375.0	2.87
ZB	50k	0		
ZN	30k	-2.5k		
ZS	50k	-2k		

Tabla 4.2: Tabla resumen de los 13 futuros evaluados en las cuatro modalidades operativas. Desde 2026-03-27 hasta 2026-06-01, aunque Robotrader empezó el 6 para tener datos para las primeras predicciones

En ganancias absolutas séptimo/noveno en la competición de 58 personas.

Conclusiones y líneas futuras

5.1 Conclusiones

Como conclusiones finales de este proyecto, estoy muy satisfecho con lo que he desarrollado en el transcurso de estos meses. Además, viendo los resultados obtenidos se puede concluir que el sistema ha funcionado como se esperaba.

He podido aplicar los conocimientos que he adquirido durante estos 4 años de carrera aunque hayan habido teoría de asignaturas que me gustaría haber podido aplicar que se han quedado fuera por falta de tiempo. Aún así también he podido ampliar mis conocimientos sobre diseño de redes neuronales.

5.2 Líneas futuras

Preguntar si demasiadas líneas futuras demuestra poco o mucho interés en el trabajo

Por limitaciones del tiempo dejo pendiente varias mejoras e implementaciones nuevas

Lo primero y de lo que se ha hablado en el desarrollo, es la implementación de modelos no supervisados en diferentes puntos del pipeline, por una parte como feature extra, por otra parte como segmentador de regímenes de mercado y por último como agrupador de futuros.

Además, también me gustaría añadir el despliegue de MLflow y Airflow ya que son partes esenciales del trabajo como ingeniero y científico de datos, aportarían mucho valor.

A parte de estas implementaciones que se han comentado en los apartados de desarrollo también existen otras cosas que me gustaría añadir a futuro.

Por una parte añadir una interfaz gráfica independiente de grafana para tener personalización total. Durante la carrera he tenido asignaturas de desarrollo Front-End que me gustaría aplicar, serviría para mostrar más información a parte de la que permite grafana además de incluir la posibilidad de interactuar con los sistemas, poder parar o arrancar el bot, decidir la lista de activos a operar, etc.

En relación con lo anterior, me gustaría añadir en esa misma página un chatbot usando RASA o un LLM para responder preguntas a partir de toda la información disponible. RASA es mucho más sencillo pero un LLM permite mayor variedad de respuestas además de que a futuro se podría implementar una API MCP (Model Context Protocol)

Más relacionado con la carrera sería aplicar conceptos como la entropía de Shannon, las cadenas de Markov junto con las FSM (Finite State Machine) y el análisis de sentimientos usando NLP con BERT y MarIA

Entropía de Shannon y teoría de la información

La entropía mide la incertidumbre del mercado

Cadenas de Markov y FSM

Modelan la probabilidad de transición entre estados de mercado, las FSM estructuran el comportamiento del bot

Análisis de sentimientos

Permiten incluir información textual de noticias y redes sociales como adición a indicadores cuantitativos.

Además de todo lo ya mencionado a continuación quiero listar una serie de mejoras que se me han ido ocurriendo al final del trabajo y durante mis jornadas estando de prácticas

Great Expectations

Como complemento a la validación manual de los datos

Engine/pipeline

Durante mis prácticas pude observar cómo trabajaban en un pipeline real de datos en un banco, se organizaban como un pipeline, cada paso es una función única y todas comparten un contexto común

Otros productos financieros

Aunque ya se justificó por qué le decidió elegir los futuros aún así se puede experimentar con otros productos como las acciones, opciones, etc. Además, se podría optar a usar los datos de los otros productos como features

Mayor nivel de profundidad de datos

Aunque los datos históricos son solo OHLCV, IB tiene más información, de la que destaca es la diferenciación de los precios de bid y ask



Figura 5.1: Añadir img con algunos plots de las características más usadas como modelos solos

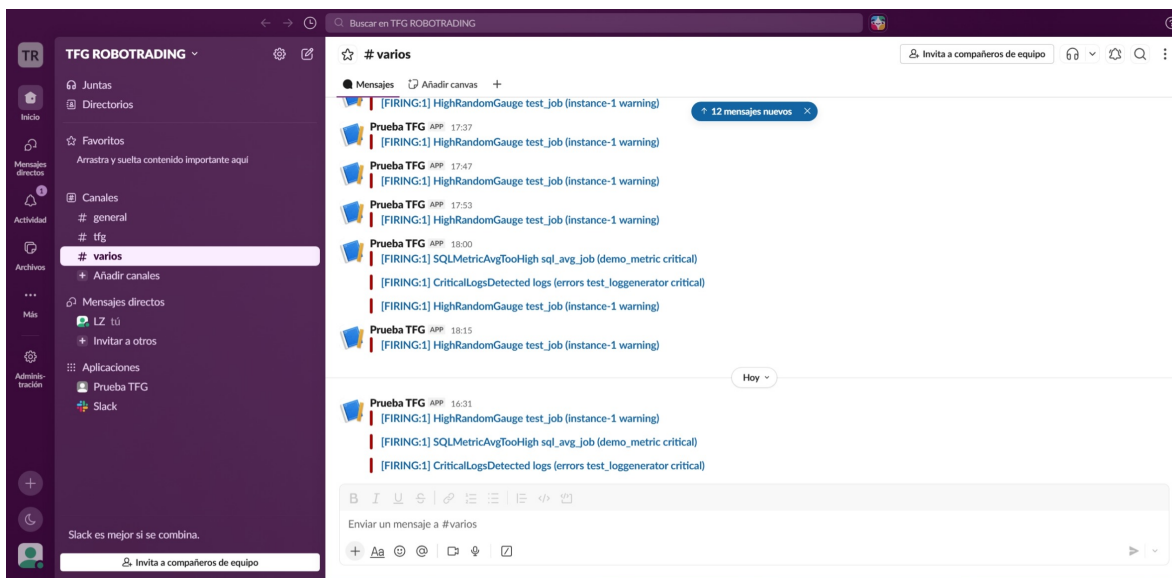


Figura 5.2: Añadir img con algunos plots de las características más usadas como modelos solos

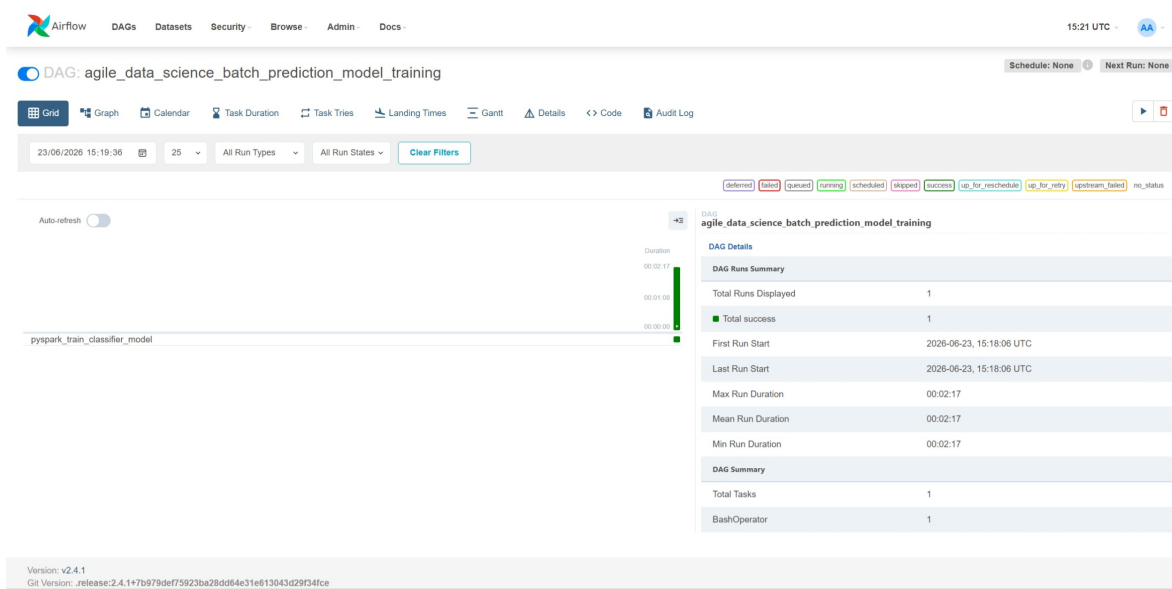


Figura 5.3: Añadir img con algunos plots de las características más usadas como modelos solos

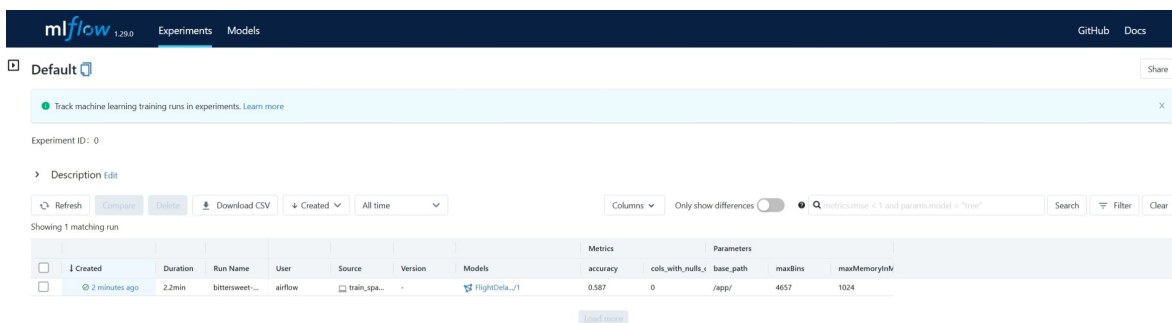


Figura 5.4: Añadir img con algunos plots de las características más usadas como modelos solos

Aspectos éticos, económicos, sociales y ambientales

6.1 Introducción

Desde el punto de vista económico el sistema tiene influencia en la reducción de costes necesarios para operar una cartera de activos, permitiendo a profesionales autónomos o quienes lo toman como un hobby. En el ámbito ético y social, . Y por último, con respecto al impacto ambiental, este sistema no tiene casi impacto al poder ejecutarse en local en hardware particular no profesional.

6.2 Descripción de impactos relevantes relacionados con el proyecto

Impacto económico

1

Impacto ético y social

2

Impacto ambiental

3

6.3 Análisis detallado de alguno de los principales impactos

El impacto mayoritario de este proyecto se centra en el tema económico

6.4 Conclusiones

Como conclusión final

Presupuesto económico

COSTE DE MANO DE OBRA (coste directo)			Horas	Precio/hora	Total
			360	25 €	9.000 €
COSTE DE RECURSOS MATERIALES (coste directo)	Precio de compra	Uso en meses	Amortización (en años)	Total	
Ordenador personal (Software incluido)	1.500,00 €	6	5	150,00 €	
COSTE TOTAL DE RECURSOS MATERIALES					150,00 €
GASTOS GENERALES (costes indirectos)	15 %	sobre CD			1372,50 €
BENEFICIO INDUSTRIAL	6 %	sobre CD+CI			631,35 €
SUBTOTAL PRESUPUESTO					11.153,85 €
IVA APLICABLE				21 %	2.342,3085 €
TOTAL PRESUPUESTO					13.496,1585 €

Bibliografía

- [1] F. Love, “Nntikz: Tikz diagrams for deep learning and neural networks,” 2024.
- [2] I. Trullàs, *Trading algorítmico con Python: Desarrolla tus habilidades en el Trading Algorítmico. Con ejemplos e integración a MetaTrader5*. Amazon Kindle Direct Publishing, 2024. Kindle edition.
- [3] D. E. Bowden, *Safety in the Market: The Smarter Starter Pack*. Queensland, Australia: Safety in the Market, third revised edition ed., 1997. ASIN: B000LYUHJE.
- [4] J. L. Elman, “Finding structure in time,” *Cognitive Science*, vol. 14, no. 2, pp. 179–211, 1990.
- [5] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [6] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, “Learning phrase representations using rnn encoder–decoder for statistical machine translation,” *arXiv preprint arXiv:1406.1078*, 2014.

Anexos

A Resultados de los entrenamientos

B Resultados de los backtest individuales

C Repositorio de GitHub con el código

El código completo desarrollado para este trabajo se encuentra disponible en el siguiente repositorio de GitHub:

<https://github.com/LijunZhou000/ML-Algorithmic-Trading>